

**UNIVERSITAS GUNADARMA
FAKULTAS TEKNOLOGI INDUSTRI**



**PENULISAN ILMIAH
DALSILENS: ALAT BANTU MOBILE REAL-TIME UNTUK
PENDERITA DIKROMASI MENGGUNAKAN DALTONISASI
LMS DAN IDENTIFIKASI WARNA BERBASIS HSV**

Nama : Anaf Naufalian
NPM : 50423155
Program Studi : Informatika
Pembimbing : Dr. Bonang Waspadadi Ligar, S.Si., MMSI

**JAKARTA
2026**

PERNYATAAN ORISINALITAS DAN PUBLIKASI

Saya yang bertanda tangan di bawah ini,

Nama : Anaf Naufalian
NPM : 50423155
Judul PI : Dalsilens: Alat Bantu Mobile Real-Time untuk Penderita Dikromasi Menggunakan Daltonisasi LMS dan Identifikasi Warna Berbasis HSV
Tanggal Sidang : 01 Januari 2026
Tanggal Lulus : 01 Januari 2026

menyatakan bahwa tulisan ini adalah merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apa pun telah mengikuti kaidah, etika yang berlaku. Mengenai isi dan tulisan adalah merupakan tanggung jawab Penulis, bukan Universitas Gunadarma. Demikian pernyataan ini dibuat dengan sebenarnya dan dengan penuh kesadaran.



Jakarta, 01 Januari 2026



(Anaf Naufalian)

LEMBAR PENGESAHAN

Judul PI : Dalsilens: Alat Bantu Mobile Real-Time untuk Penderita Dikromasi Menggunakan Daltonisasi LMS dan Identifikasi Warna Berbasis HSV
Nama : Anaf Naufalian
NPM : 50423155
Program Studi : Informatika
Tanggal Sidang : 01 Januari 2026
Tanggal Lulus : 01 Januari 2026

Menyetujui

Pembimbing

Kasubbag. Sidang PI

(Dr. Bonang Waspadadi Ligar, S.Si.,
MMSI)

(Dr. Achmad Fahrurozi, S.Si., M.Si.)

Ketua Program Studi

(Prof. Dr. Lintang Yuniar Banowosari, S.Kom., M.Sc.)

ABSTRAK

Anaf Naufalian, 50423155

DALSILENS: ALAT BANTU MOBILE REAL-TIME UNTUK PENDERITA DIKROMASI MENGGUNAKAN DALTONISASI LMS DAN IDENTIFIKASI WARNA BERBASIS HSV

Penulisan Ilmiah. Informatika. Fakultas Teknologi Industri. Universitas Gunadarma. 2026

Kata Kunci: Android, Daltonisasi, Dikromasi, Pemrosesan Citra Real-Time, Ruang Warna LMS

(xi + 54 + Lampiran)

Color Vision Deficiency (CVD), atau yang dikenal sebagai buta warna merupakan kondisi kelainan genetik yang memengaruhi kemampuan seseorang dalam membedakan panjang gelombang cahaya tertentu. Meskipun ketajaman penglihatan penderita umumnya normal, individu dengan CVD mengalami kesulitan membedakan warna dalam spektrum merah, hijau, dan biru. Tipe yang paling umum adalah Dikromasi, yang mencakup *Protanopia* (buta warna merah), *Deutanopia* (buta warna hijau), dan *Tritanopia* (buta warna biru). Solusi bantu yang sudah ada, seperti kacamata penyaring warna, dirancang untuk memisahkan panjang gelombang merah dan hijau yang tumpang tindih guna membantu pembedaan warna, namun harganya sering kali mahal. Selain itu, banyak aplikasi *smartphone* yang ada mengharuskan pengguna mengunggah gambar untuk diproses, sehingga tidak dapat berjalan secara *real-time* dan membatasi umpan balik visual secara langsung. Penulisan ini mengusulkan implementasi algoritma *LMS Daltonization* pada platform Android menggunakan pemrosesan citra secara *real-time*. Metode ini mengonversi masukan dari kamera dari ruang warna RGB ke ruang warna LMS. Hal ini memungkinkan simulasi matematis dan penyesuaian spektrum warna yang hilang. Aplikasi ini diharapkan dapat membantu pengguna membedakan objek dengan spektrum warna tertentu melalui *smartphone*, dengan latensi rendah yang mendukung penggunaan secara *real-time*.

Daftar Pustaka (1995 – 2026)

KATA PENGANTAR

Segala puji dan syukur ke hadirat Tuhan Yang Maha Kuasa yang telah memberikan berkat, anugerah dan karunia yang melimpah, sehingga penulis dapat menyelesaikan Penulisan Ilmiah ini. Penulisan Ilmiah ini disusun guna melengkapi sebagian syarat dalam mencapai gelar Setara Sarjana Muda pada Program Studi Informatika, Fakultas Teknologi Industri, Universitas Gunadarma. Adapun judul Penulisan Ilmiah ini adalah **“Dalsilens: Alat Bantu Mobile Real-Time untuk Penderita Dikromasi Menggunakan Daltonisasi LMS dan Identifikasi Warna Berbasis HSV”**.

Walaupun banyak kesulitan yang penulis harus hadapi ketika menyusun Penulisan Ilmiah ini, namun berkat bantuan dan dorongan dari berbagai pihak akhirnya tugas ini dapat diselesaikan dengan baik. Untuk itu penulis mengucapkan terima kasih, kepada:

1. Prof. Dr. E.S. Margianti, S.E., M.M., selaku Rektor Universitas Gunadarma.
2. Prof. Dr. -Ing. Adang Suhandra, S.Si., S.Kom., M.Sc., selaku Dekan Fakultas Teknologi Industri Universitas Gunadarma.
3. Prof. Dr. Lintang Yuniar Banowosari, S.Kom., M.Sc., selaku Ketua Program Studi Informatika Universitas Gunadarma.
4. Dr. Achmad Fahrurozi, S.Si., M.Si., selaku Kepala Sub Bagian Sidang Penulisan Ilmiah Fakultas Teknologi Industri Universitas Gunadarma.
5. Dr. Bonang Waspadadi Ligar, S.Si., MMSI, selaku Dosen Pembimbing yang telah banyak memberikan bimbingan dan arahan kepada penulis dalam menyelesaikan Penulisan Ilmiah ini.
6. Kedua orang tua dan keluarga penulis, yang senantiasa memberikan doa, dukungan, dan motivasi tanpa henti selama proses penyusunan Penulisan Ilmiah ini.

7. Seluruh teman dan sahabat penulis, yang telah memberikan bantuan, semangat, dan masukan yang berarti selama penyusunan Penulisan Ilmiah ini.

Penulis menyadari bahwa Penulisan Ilmiah ini masih jauh dari sempurna, sehingga saran dan kritik yang membangun dari berbagai pihak sangat diharapkan demi penyempurnaan tulisan ini.

Jakarta, Januari 2026

A handwritten signature in black ink, consisting of a large, stylized 'A' followed by a horizontal line and a small loop.

(Anaf Naufalian)

DAFTAR ISI

PERNYATAAN ORISINALITAS DAN PUBLIKASI	ii
LEMBAR PENGESAHAN	iii
ABSTRAK	iv
KATA PENGANTAR	v
DAFTAR ISI	vii
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
DAFTAR LAMPIRAN	xi
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Ruang Lingkup	3
1.4 Tujuan Penelitian	4
1.5 Metode Penelitian	4
1.6 Sistematika Penulisan Ilmiah	5
2 TINJAUAN PUSTAKA	7
2.1 Penelitian Terkait	7
2.2 Konsep Dasar Penglihatan dan Warna	8
2.2.1 <i>Color Vision Deficiency</i> (CVD) dan Dikromasi	8
2.2.2 Ruang Warna Visual (RGB, LMS, dan HSV)	8
2.3 Pengolahan Citra Digital (<i>Digital Image Processing</i>)	8
2.3.1 Pemrosesan Citra <i>Real-Time</i>	8
2.3.2 Algoritma LMS Daltonization	9
2.4 Teknologi dan Perangkat Pengembang	9
2.4.1 Sistem Operasi Android	9
2.4.2 Pustaka Grafis <i>Color Matrix</i> dan Akselerasi Perangkat Keras	10
2.4.3 Bahasa Pemrograman	11
2.4.4 Metode Pengembangan Perangkat Lunak (SDLC)	11
2.4.5 Unified Modeling Language (UML)	12
2.5 Metode Pengujian Sistem	13
2.5.1 Pengujian Fungsionalitas (<i>Black Box Testing</i>)	13
2.5.2 Pengujian Akurasi Klasifikasi Warna	13
2.5.3 Pengujian Performa <i>Real-Time</i>	14
2.5.4 Pengujian Kebergunaan (<i>System Usability Scale</i>)	14
3 PERANCANGAN DAN IMPLEMENTASI	15
3.1 Gambaran Umum Sistem	15
3.1.1 Metode Pengembangan Perangkat Lunak	15
3.2 Arsitektur Sistem	18
3.2.1 Perancangan UML	19
3.2.2 Pengambilan Gambar dan Identifikasi Warna	26
3.2.3 Proses Daltonisasi	27
3.3 Implementasi Algoritma <i>LMS Daltonization</i>	29
3.3.1 Konversi Ruang Warna RGB ke LMS	29

3.3.2	Simulasi Buta Warna	29
3.3.3	Proses Daltonisasi	30
3.3.4	Konstruksi Matriks Transformasi 4×5	32
3.4	Implementasi Tingkat Keparahan (<i>Severity Level</i>)	33
3.4.1	Keparahan pada Mode Simulasi	33
3.4.2	Keparahan pada Mode Daltonisasi	34
3.5	Ekstraksi Warna Dominan Berbasis HSV	34
3.5.1	Pengambilan <i>Bitmap</i> dari Kamera	36
3.5.2	Pengambilan Sampel Piksel	36
3.5.3	Klasifikasi Nama Warna	37
3.6	Skenario Pengujian	39
3.6.1	Pengujian Akurasi Klasifikasi Nama Warna	39
3.6.2	Pengujian Performa <i>Real-Time</i>	41
3.6.3	Pengujian Fungsionalitas (<i>Black Box Testing</i>)	46
3.6.4	Pengujian Kebergunaan (<i>System Usability Scale</i>)	48
4	PENUTUP	52
4.1	Kesimpulan	52
4.2	Saran	53
	DAFTAR PUSTAKA	55
	LAMPIRAN	L-1

DAFTAR TABEL

3.1	Klasifikasi Warna Berdasarkan Rentang <i>Hue</i>	38
3.2	Klasifikasi Warna Netral Berdasarkan <i>Saturation</i> dan <i>Value</i>	38
3.3	Hasil Pengujian Klasifikasi Warna pada Pencahayaan Normal	40
3.4	Hasil Pengujian Klasifikasi Warna pada Pencahayaan Redup	40
3.5	Hasil Pengujian FPS dan Latensi Pemrosesan	42
3.6	Hasil Pengujian Penggunaan Memori (RAM)	44
3.7	Hasil Pengujian Utilisasi CPU	45
3.8	Hasil Pengujian <i>Black Box Testing</i> Aplikasi Dalsilens	46
3.9	Klasifikasi Skor <i>System Usability Scale</i>	49
3.10	Data Responden Pengujian SUS	50
3.11	Rekapitulasi Jawaban Responden Kuesioner SUS	51

DAFTAR GAMBAR

3.1	Gambaran Umum Arsitektur Sistem Dalsilens	16
3.2	Tahapan Model <i>Waterfall</i> pada Pengembangan Dalsilens	17
3.3	<i>Use Case Diagram</i> Aplikasi Dalsilens	19
3.4	<i>Activity Diagram</i> Pengambilan Gambar dan Identifikasi Warna	22
3.5	<i>Activity Diagram</i> Proses Daltonisasi	23
3.6	<i>Class Diagram</i> Aplikasi Dalsilens	24
3.7	Diagram Alur Pengambilan Gambar dan Identifikasi Warna	27
3.8	Diagram Alur Simulasi Buta Warna dan Proses Daltonisasi	28
3.9	Diagram Alur Ekstraksi dan Klasifikasi Warna Secara <i>Real-Time</i> . . .	35

DAFTAR LAMPIRAN

Lampiran 1	Pernyataan Uji Coba Aplikasi	L-1
Lampiran 2	Listing Program	L-2
Lampiran 3	Output Program	L-17

1. PENDAHULUAN

1.1. Latar Belakang

Penglihatan merupakan indra utama pada manusia yang berperan dalam memproses informasi dari lingkungan eksternal. Beberapa penelitian menyebutkan bahwa sekitar 80% (Man dan Olchawa, 2018) informasi yang diterima dan diproses oleh otak manusia berasal dari indera penglihatan (sistem visual). Hal ini menunjukkan bahwa indra penglihatan berperan sangat penting untuk memahami lingkungan sekitar (Chung dan Son, 2020). Namun, tidak semua orang memiliki kemampuan ini karena adanya *Color Vision Deficiency* (Defisiensi Penglihatan Warna) atau buta warna. Penderita buta warna (khususnya *Protanopia* dan *Deutanopia*) sering kali mengalami kesulitan dalam membedakan warna tertentu seperti merah (*Protanopia*) dan hijau (*Deutanopia*) (Lin, Chen dan Wang, 2019). Kesulitan ini dapat mempengaruhi kemampuan individu untuk membedakan warna seperti lampu lalu lintas, kode warna pada perangkat elektronik, hingga cairan.

Untuk mengatasi keterbatasan identifikasi visual tersebut, penggunaan teknologi dapat menjadi alternatif solusi yang cukup efektif. Seiring berkembangnya teknologi, telepon pintar (*smartphone*) kini memiliki kemampuan komputasi yang tinggi untuk melakukan pemrosesan citra digital (*Digital Image Processing*) secara *real-time*. Melalui pemrosesan citra, kelemahan penglihatan warna visual dapat dikoreksi menggunakan algoritma khusus seperti *LMS Daltonization*, algoritma ini bekerja dengan cara memanipulasi matriks warna untuk menggeser informasi warna yang hilang ke area yang masih dapat dilihat oleh penderita. Namun, memanipulasi warna pada setiap frame secara terus menerus memerlukan sumber daya komputasi (*computational resources*) yang besar, sehingga dibutuhkan pemrosesan grafis yang efisien. Oleh karena itu, sistem operasi Android dipilih sebagai platform pengembangan. Selain mendominasi pasar perangkat seluler secara global dengan persentase sebesar 71.68% pada tahun 2025 (StatCounter, 2025), Android menyediakan dukungan

pustaka (*library*) bawaan yaitu *Color Matrix*. Pustaka ini memungkinkan proses kalkulasi algoritma *LMS Daltonization* pada tingkat piksel dieksekusi secara langsung melalui akselerasi perangkat keras (*hardware acceleration*), salah satunya *Graphical Processing Unit* (GPU), sehingga performa perangkat tetap stabil tanpa membebani *Central Processing Unit* (CPU). Selain itu, kemampuan *smartphone* dalam mengekstraksi nilai piksel dari tangkapan kamera juga membuka peluang penerapan metode komputasi pada ruang warna HSV (*Hue, Saturation, Value*), yang memungkinkan sistem untuk mengklasifikasikan nama warna di lingkungan sekitar secara dinamis.

Berbagai solusi untuk permasalahan ini sebenarnya sudah tersedia, salah satunya adalah kacamata kromatik yang dirancang secara khusus untuk menyaring panjang gelombang cahaya tertentu yang tumpang tindih antara warna merah dan hijau (Patterson, Mastey, Kuchenbecker, Rowlan, J. Neitz, M. Neitz dan Carroll, 2022). Namun, penggunaan kacamata kromatik masih memiliki beberapa keterbatasan, seperti harganya yang relatif tinggi dan ketersediaannya yang terbatas.

Mengatasi keterbatasan tersebut, penelitian ini mengusulkan solusi alternatif dengan merancang dan mengimplementasikan sebuah aplikasi alat bantu visual berjudul “Dalsilens: Alat Bantu Mobile Real-Time untuk Penderita Dikromasi Menggunakan Daltonisasi LMS dan Identifikasi Warna Berbasis HSV“. Aplikasi ini dirancang untuk membantu penyandang buta warna dalam membedakan warna sekaligus mengidentifikasi nama warna pada suatu objek dengan memanfaatkan kamera *smartphone* secara *real-time*. Solusi ini menggunakan algoritma *LMS Daltonization* untuk menggeser informasi warna yang hilang ke spektrum warna lain, serta pendekatan *rule-based thresholding* pada ruang warna HSV untuk klasifikasi nama warna. Dengan pendekatan berbasis perangkat lunak ini, Dalsilens diharapkan dapat menjadi alat bantu yang praktis, memiliki latensi rendah, serta lebih mudah dijangkau dibandingkan solusi optik konvensional.

1.2. Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Bagaimana merancang dan mengimplementasikan aplikasi Android yang dapat membantu penderita dikromasi membedakan warna secara visual menggunakan algoritma *LMS Daltonization* secara *real-time*?
2. Bagaimana mengimplementasikan fitur deteksi nama warna berbasis ruang warna HSV yang mampu mengklasifikasikan warna objek di lingkungan sekitar secara otomatis?
3. Bagaimana tingkat fungsionalitas, akurasi, performa, dan kebergunaan aplikasi Dalsilens berdasarkan hasil pengujian?

1.3. Ruang Lingkup

Adapun ruang lingkup pada penulisan ini yaitu:

1. Aplikasi dikembangkan untuk perangkat dengan sistem operasi Android menggunakan bahasa Kotlin.
2. Kondisi defisiensi penglihatan warna yang ditangani difokuskan pada tipe Dikromasi, yang mencakup *Protanopia* (buta warna merah), *Deutanopia* (buta warna hijau), dan *Tritanopia* (buta warna biru).
3. Penelitian ini tidak mencakup kondisi Monokromasi (buta warna total).
4. Akurasi dari fitur klasifikasi nama warna juga dipengaruhi oleh kondisi pencahayaan lingkungan sekitar.
5. Pengujian koreksi warna dilakukan secara fungsional menggunakan metode *Black Box Testing*, bukan uji klinis medis langsung terhadap pasien.

1.4. Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Pengembangan prototipe aplikasi seluler Dalsilens berbasis Android sebagai alat bantu visual secara *real-time* bagi penderita dikromasi.
2. Implementasi algoritma LMS Daltonization untuk membedakan objek secara visual.
3. Implementasi deteksi nama warna berbasis ruang warna HSV untuk membedakan objek secara tekstual.

1.5. Metode Penelitian

Metode penelitian ini dilakukan melalui beberapa tahapan. Tahap pertama dimulai dengan perancangan arsitektur sistem untuk mengambil citra secara *real-time*. Selanjutnya, algoritma *LMS Daltonization* akan diimplementasikan untuk mengonversi ruang warna dan menyesuaikan piksel ke spektrum yang mudah dikenali oleh pengguna. Setelah itu, dilakukan ekstraksi warna dominan menggunakan ruang warna HSV dengan pendekatan *rule-based thresholding* untuk mengklasifikasikan nama warna. Tahapan terakhir adalah pengujian sistem untuk mengevaluasi fungsionalitas dan performa aplikasi.

Untuk mendukung seluruh tahap penelitian, perangkat keras (*hardware*) dan perangkat lunak (*software*) akan digunakan. Perangkat keras yang digunakan meliputi komputer atau laptop untuk pengembangan aplikasi, serta *smartphone* berbasis Android sebagai alat untuk pengujian, khususnya untuk pemrosesan kamera secara langsung. Sementara itu, perangkat lunak yang digunakan meliputi Android Studio sebagai *Integrated Development Environment* (IDE) untuk pengembangan aplikasi, serta pustaka bawaan seperti *CameraX* dan *ColorMatrix* untuk pengolahan grafis.

Adapun rincian dari langkah-langkah penelitian di atas adalah sebagai berikut:

1. Perancangan Arsitektur Sistem: Tahapan ini berfokus pada perancangan alur kerja (*workflow*) aplikasi agar mampu memproses citra secara *real-time* dengan latensi yang rendah. Komputasi sistem dibagi menjadi dua tahapan utama, yaitu pengambilan gambar dari komponen kamera dan manipulasi *bitmap* menggunakan *color matrix* yang dioptimalkan melalui akselerasi GPU.
2. Implementasi Algoritma LMS Daltonization: Pada tahap ini akan diterapkan model matematika untuk mengonversi ruang warna RGB menjadi ruang warna LMS. Sistem akan mensimulasikan defisiensi buta warna, menghitung informasi warna yang hilang, lalu menggeser (*error shifting*) informasi warna tersebut ke spektrum warna lain yang kemudian diintegrasikan ke dalam matriks transformasi.
3. Ekstraksi Warna Dominan Berbasis HSV: Tahap ini menjelaskan mekanisme pengambilan sampel piksel pada titik tengah layar. Citra *bitmap* diekstrak nilai rata-rata RGB-nya, dikonversi ke ruang warna HSV, lalu diklasifikasikan nama warnanya menggunakan metode *rule-based thresholding*.
4. Skenario Pengujian Sistem: Tahap terakhir adalah evaluasi untuk mengukur fungsionalitas dan performa aplikasi. Pengujian dibagi menjadi empat skenario yaitu pengujian fungsionalitas menggunakan metode *Black Box Testing*, uji akurasi deteksi nama warna, uji kinerja komputasi *real-time* pada perangkat, dan uji kebergunaan menggunakan metode *System Usability Scale* (SUS).

1.6. Sistematika Penulisan Ilmiah

1. BAB I: PENDAHULUAN

Bab ini menjelaskan latar belakang masalah mengenai defisiensi penglihatan warna (dikromasi) dan keterbatasan solusi yang ada. Selain itu, dibahas juga rumusan masalah, ruang lingkup penelitian, tujuan, metode penelitian, dan sistematika penulisan.

2. **BAB II: LANDASAN TEORI**

Bab ini berisi tinjauan pustaka dan teori yang mendukung penelitian, seperti konsep *Color Vision Deficiency* (CVD), pengolahan citra digital, algoritma LMS Daltonization, ruang warna HSV, serta teknologi Android dan pustaka grafis yang digunakan.

3. **BAB III: PERANCANGAN DAN IMPLEMENTASI**

Bab ini menjelaskan tahapan penelitian, perangkat yang digunakan, serta perancangan sistem. Selain itu, dijelaskan juga alur implementasi penyesuaian warna, ekstraksi warna dominan, beserta hasil pengujian fungsionalitas dan performa pada aplikasi Dalsilens.

4. **BAB IV: PENUTUP**

Bab ini berisi kesimpulan dari hasil penelitian serta saran untuk pengembangan aplikasi Dalsilens ke depan.

2. TINJAUAN PUSTAKA

2.1. Penelitian Terkait

Dalam beberapa tahun terakhir berbagai solusi berbasis teknologi telah banyak dikembangkan untuk membantu individu dengan *Color Vision Deficiency* (CVD) untuk meningkatkan kemampuan dalam membedakan warna. Penelitian yang dilakukan oleh Elrefaei (2018) melakukan perbandingan empat algoritma koreksi warna berbasis *smartphone*, termasuk algoritma *LMS Daltonization*, yang dievaluasi menggunakan software simulator Vischeck dan Coblis. Namun, pengujian pada penelitian tersebut memiliki keterbatasan karena pemrosesan citra masih berbasis statis dengan metode unggah gambar secara manual dan tidak berjalan secara *real-time*.

Selain itu, penelitian lain oleh Tecson, Calanda, Cayabyab dan Reyes Jr (2017) mengembangkan aplikasi seluler Covisance yang menerapkan metode LMS Daltonization secara *real-time* dan mengintegrasikan uji Farnsworth D-15 langsung di dalam aplikasi. Meskipun mampu menampilkan simulasi secara langsung, aplikasi tersebut masih menggunakan metode kalkulasi nilai warna secara manual, yang dapat membebani kinerja CPU dan sering menyebabkan frame-drop pada perangkat dengan resolusi kamera tinggi.

Oleh karena itu, penelitian ini berfokus pada efisiensi komputasi visual secara *real-time* dengan cara memindahkan beban komputasi dari CPU ke GPU untuk menghindari proses sekuensial yang berat. Dengan memanfaatkan *library* CameraX dan *ColorMatrix*, Dalsilens dapat mempertahankan stabilitas koreksi warna pada pemrosesan *real-time*, sehingga dapat mengurangi masalah penurunan bingkai (*frame drop*) dan latensi yang sering terjadi pada pemrosesan citra *real-time*.

2.2. Konsep Dasar Penglihatan dan Warna

2.2.1. *Color Vision Deficiency (CVD) dan Dikromasi*

Color Vision Deficiency (CVD) atau buta warna adalah sebuah kondisi kelainan genetik yang memengaruhi kemampuan seseorang dalam membedakan panjang gelombang cahaya tertentu seperti merah, hijau, dan biru. Tipe paling umum dari kondisi ini adalah Dikromasi, yang terbagi menjadi *Protanopia* (buta warna merah), *Deutanopia* (buta warna hijau), dan *Tritanopia* (buta warna biru) (Muhammad, Maitama, Bature dan Yunusa, 2020).

2.2.2. Ruang Warna Visual (RGB, LMS, dan HSV)

Umumnya pemrosesan citra digital menggunakan ruang warna RGB (*Red, Green, Blue*). Namun, sistem penglihatan manusia memproses spektrum cahaya melalui sel kerucut mata yang direpresentasikan dalam ruang warna LMS (*Long, Medium, Short*) (Tasnim dan Hasan, 2017). Selain itu, terdapat ruang warna HSV (*Hue, Saturation, Value*) yang efektif dalam pemisahan intensitas cahaya dari warna itu sendiri. Ruang warna ini membagi elemen visual menjadi tipe jenis warna (H), intensitas warna (S), dan tingkat kecerahan (V), sehingga cocok digunakan untuk klasifikasi warna (Y. Sirisathitkul, Dinmeung, Noonsuk dan C. Sirisathitkul, 2025).

2.3. Pengolahan Citra Digital (*Digital Image Processing*)

2.3.1. Pemrosesan Citra *Real-Time*

Pengolahan citra digital merupakan teknik untuk memproses dan memodifikasi gambar digital dengan mengubah nilai piksel melalui proses komputasi. Pengolahan citra digital secara *real-time* mengharuskan sistem untuk terus menerus mengambil sampel bingkai (*frame*) dari perangkat keras kamera. Nilai piksel dari bingkai yang ditangkap tersebut kemudian diekstrak untuk diproses dan dianalisis sebelum ditampilkan kembali kepada pengguna secara *real-time* (Samyan, Sahar, Talha, Aslam dan Martinez-Enriquez, 2015).

2.3.2. Algoritma LMS Daltonization

LMS Daltonization adalah sebuah teknik pemrosesan gambar untuk memanipulasi warna pada citra, yang dapat membantu individu dengan buta warna supaya dapat membedakan warna tertentu (Muhammad, Maitama, Bature dan Yunusa, 2020). Secara garis besar metode ini dibagi menjadi beberapa tahap, tahap pertama yaitu mengonversi ruang warna RGB menjadi ruang warna LMS (Elrefaei, 2018; Tecson, Calanda, Cayabyab dan Reyes Jr, 2017). Tahap kedua adalah mensimulasikan buta warna (*color blindness*), pada tahap ini salah satu sel komponen (L, M, atau S) dimodifikasi untuk dihitung ulang berdasarkan kombinasi dua komponen lainnya untuk menggambarkan warna apa yang sebenarnya tertangkap oleh mata individu yang mengalami buta warna. Tahap ketiga yaitu menghitung nilai *error* (informasi warna yang hilang), hasil simulasi pada tahap sebelumnya kemudian dikonversi kembali dari ruang warna LMS ke ruang warna RGB. Sistem lalu menghitung selisih atau *error* antara nilai RGB asli dengan nilai RGB hasil simulasi. Nilai *error* ini merepresentasikan informasi warna yang tidak bisa dilihat oleh individu yang mengalami buta warna. Tahap terakhir yaitu membentuk citra baru (*Daltonized*), nilai *error* yang didapat dari tahap sebelumnya ditambahkan ke warna asli. Hasil yang didapat adalah citra baru yang telah disesuaikan sehingga individu dengan buta warna dapat melihat perbedaan warna tertentu, menjadi lebih mudah dikenali tanpa menghilangkan keseluruhan informasi warna pada citra asli.

2.4. Teknologi dan Perangkat Pengembang

2.4.1. Sistem Operasi Android

Android merupakan sebuah sistem operasi seluler yang bersifat terbuka (*open-source*) dan dibangun di atas kernel Linux. Android juga dipilih karena memiliki ekosistem pengembangan yang matang serta didukung oleh komunitas pengembang yang besar (Francis, 2023; Stack Overflow, 2025). Selain itu dominasi Android di pasar global juga sangat besar, di mana data *StatCounter* mencatat bahwa platform ini menguasai pangsa pasar perangkat seluler sebesar 71,68% secara global pada

tahun 2025 (StatCounter, 2025).

Dalam konteks pengembangan aplikasi alat bantu visual *real-time*, platform Android menawarkan berbagai keuntungan, terutama melalui ketersediaan *Application Programming Interface* (API) yang luas untuk mengakses komponen perangkat keras seperti kamera menggunakan pustaka *CameraX* (Android Developers, 2024). Dukungan ini sangat penting dalam membangun sistem bantu visual yang memerlukan proses tangkapan citra secara terus-menerus.

2.4.2. Pustaka Grafis *Color Matrix* dan Akselerasi Perangkat Keras

Color Matrix adalah sebuah pustaka grafis bawaan pada pemrograman Android yang difungsikan untuk memodifikasi representasi warna citra digital. Pustaka ini bekerja dengan menerapkan transformasi matematika melalui perkalian matriks berukuran 4×5 terhadap nilai *Red*, *Green*, *Blue*, dan *Alpha* (RGBA) pada setiap piksel. Dalam konteks pemrosesan visual dikromasi, pustaka ini cocok digunakan karena proses perhitungan pada algoritma *LMS Daltonization* dapat diterapkan dalam bentuk operasi matriks warna (Android Developers, 2026b).

Keunggulan utama dari penggunaan pustaka ini adalah dukungannya terhadap akselerasi perangkat keras (*Hardware Acceleration*). Melalui fitur ini, beban manipulasi grafis tingkat piksel yang berat dialihkan dari *Central Processing Unit* (CPU) ke *Graphics Processing Unit* (GPU). Hal ini memberikan keuntungan berupa peningkatan efisiensi komputasi, sehingga aplikasi mampu merender koreksi warna secara *real-time* tanpa mengalami *frame-drop* atau latensi tinggi. Walaupun sangat efisien, keterbatasan dari pemanfaatan teknologi ini adalah performa yang pada akhirnya tetap bergantung pada kapabilitas *chipset* grafis masing-masing perangkat, di mana komputasi bingkai (*frame*) yang terlalu berat pada perangkat berspesifikasi rendah tetap memiliki potensi membuat perangkat menjadi lebih panas (Android Developers, 2026c).

2.4.3. Bahasa Pemrograman

Pada penelitian ini, pengembangan perangkat lunak menggunakan bahasa Kotlin. Kotlin adalah bahasa pemrograman modern berorientasi objek yang dikembangkan oleh perusahaan JetBrains. Sejak tahun 2019, Google telah meresmikan Kotlin sebagai bahasa utama yang paling direkomendasikan untuk pengembangan aplikasi Android (Android Developers, 2026a).

Salah satu keunggulan bahasa Kotlin adalah fitur keamanan null (*null safety*). Fitur ini berfungsi untuk mencegah terjadinya *NullPointerException* (NPE) pada saat kompilasi, yang merupakan kesalahan paling sering terjadi jika menggunakan bahasa pemrograman Java. Selain itu sintaks pada bahasa Kotlin dirancang lebih ringkas untuk mengurangi kode yang berulang (*boilerplate code*). Hal ini memungkinkan penulisan algoritma kompleks seperti penerapan manipulasi warna dan klasifikasi warna ditulis dengan kode yang lebih sedikit. (JetBrains, 2026)

Pada penelitian ini Kotlin dipilih karena memiliki performa yang mumpuni. Bahasa ini hampir sepenuhnya memiliki interoperabilitas dengan Java, yang berarti Kotlin dapat memanggil pustaka bawaan Java dan Android SDK (*Software Development Kit*) secara mudah.

2.4.4. Metode Pengembangan Perangkat Lunak (SDLC)

Dalam merencanakan sebuah perangkat lunak, diperlukan sebuah metodologi yang dikenal sebagai *Software Development Life Cycle* (SDLC). SDLC adalah kerangka kerja (*framework*) yang digunakan untuk mengelola proses pengembangan perangkat lunak dari tahap perencanaan, analisis, perancangan, implementasi, pengujian, hingga pemeliharaan sistem. SDLC membantu proses pengembangan perangkat lunak menjadi lebih terstruktur karena setiap tahap dilakukan secara berurutan sesuai dengan fungsinya masing-masing. Dengan mengikuti tahapan tersebut, pengembang dapat lebih mudah merancang, membangun, menguji, dan memelihara perangkat lunak sehingga aplikasi yang dihasilkan dapat berjalan sesuai dengan kebutuhan

pengguna (IBM, 2026).

2.4.5. Unified Modeling Language (UML)

Unified Modeling Language (UML) merupakan bahasa pemodelan visual yang digunakan untuk menggambarkan, merancang, dan mendokumentasikan sistem perangkat lunak. UML membantu pengembang memahami struktur dan alur kerja sistem sebelum proses pengembangan dimulai. Dalam pengembangan perangkat lunak, UML menyediakan berbagai jenis diagram yang digunakan untuk menggambarkan struktur, fungsi, dan alur kerja sistem.

Use Case Diagram

Use Case Diagram adalah diagram UML yang digunakan untuk menggambarkan interaksi antara aktor (pengguna atau sistem lain) dengan fitur-fitur yang tersedia pada sistem. Diagram ini menunjukkan fungsi-fungsi yang dapat dilakukan oleh pengguna tanpa menjelaskan proses atau detail teknis di dalam sistem. Dengan diagram ini, kebutuhan dan fungsi utama aplikasi dapat digambarkan dengan lebih jelas.

Activity Diagram

Activity Diagram digunakan untuk menggambarkan alur aktivitas atau proses yang terjadi di dalam sistem. Diagram ini menunjukkan urutan langkah-langkah yang dilakukan mulai dari awal hingga akhir proses, termasuk percabangan keputusan yang mungkin terjadi. Diagram ini membantu dalam memahami bagaimana suatu fitur atau proses berjalan di dalam aplikasi.

Class Diagram

Class Diagram merupakan diagram yang digunakan untuk menggambarkan struktur sistem berdasarkan kelas (*class*), atribut, metode, dan hubungan antar kelas. Diagram ini memberikan gambaran tentang bagaimana komponen-komponen dalam sistem saling berhubungan. Dalam pengembangan perangkat lunak berorientasi

objek, *Class Diagram* sering digunakan sebagai referensi dalam proses implementasi kode program.

2.5. Metode Pengujian Sistem

Tahap terakhir dalam pengembangan aplikasi Dalsilens adalah melakukan pengujian (*testing*) untuk memastikan aplikasi dapat berjalan dengan baik dan sesuai dengan tujuan penelitian. Penelitian ini menggunakan empat metode pengujian yang mencakup aspek fungsionalitas, akurasi, performa, dan kebergunaan.

2.5.1. Pengujian Fungsionalitas (*Black Box Testing*)

Pengujian *Black Box* dilakukan untuk memeriksa apakah fitur-fitur pada aplikasi dapat berjalan sesuai dengan kebutuhan yang telah ditentukan. Pada metode ini, pengujian dilakukan dengan mencoba berbagai masukan (*input*) dan melihat hasil yang diberikan oleh sistem tanpa memperhatikan kode program yang ada di dalamnya. Pengujian dilakukan pada fitur-fitur utama aplikasi seperti membuka kamera, mengganti mode simulasi dan daltonisasi, mengatur *severity level*, serta menampilkan hasil klasifikasi warna. Aplikasi dinyatakan berhasil apabila setiap fitur dapat memberikan keluaran yang sesuai dengan hasil yang diharapkan (Dicoding Indonesia, 2024).

2.5.2. Pengujian Akurasi Klasifikasi Warna

Pengujian akurasi dilakukan untuk mengevaluasi kemampuan fitur identifikasi nama warna dalam mengenali warna objek nyata secara *real-time*. Pengujian dilakukan dengan mengarahkan kamera ke berbagai objek berwarna dalam dua kondisi pencahayaan, yaitu normal dan redup. Hasil deteksi nama warna yang ditampilkan aplikasi kemudian dibandingkan dengan warna objek yang sebenarnya untuk menghitung persentase akurasi.

2.5.3. Pengujian Performa *Real-Time*

Pengujian performa dilakukan untuk memvalidasi efisiensi komputasi aplikasi saat menjalankan transformasi *color matrix* dan klasifikasi warna secara bersamaan. Parameter yang diukur meliputi nilai *Frame Per Second* (FPS), penggunaan memori (RAM), dan utilisasi CPU menggunakan *Android Studio Profiler*.

2.5.4. Pengujian Kebergunaan (*System Usability Scale*)

System Usability Scale (SUS) merupakan metode yang digunakan untuk mengukur tingkat kebergunaan (*usability*) suatu sistem berdasarkan persepsi pengguna. Metode ini dikembangkan oleh John Brooke pada tahun 1995 dan telah banyak digunakan untuk mengevaluasi kemudahan penggunaan berbagai jenis perangkat lunak (Brooke, 1995). Melalui metode SUS, peneliti dapat mengetahui apakah suatu aplikasi mudah dipahami, mudah digunakan, dan dapat diterima oleh pengguna.

Pada penelitian ini, pengujian SUS dilakukan dengan melibatkan 18 responden yang diminta untuk mencoba aplikasi Dalsilens dan menggunakan fitur-fitur yang tersedia. Setelah selesai menggunakan aplikasi, responden mengisi kuesioner SUS yang terdiri dari 10 pernyataan dengan skala penilaian 1 sampai 5. Nilai yang diperoleh kemudian dihitung menggunakan metode perhitungan SUS untuk mendapatkan skor akhir yang menunjukkan tingkat kebergunaan dan penerimaan pengguna terhadap aplikasi yang dikembangkan (RG Guntur Alam dan Puji Rahayu Kurniasih, 2024).

3. PERANCANGAN DAN IMPLEMENTASI

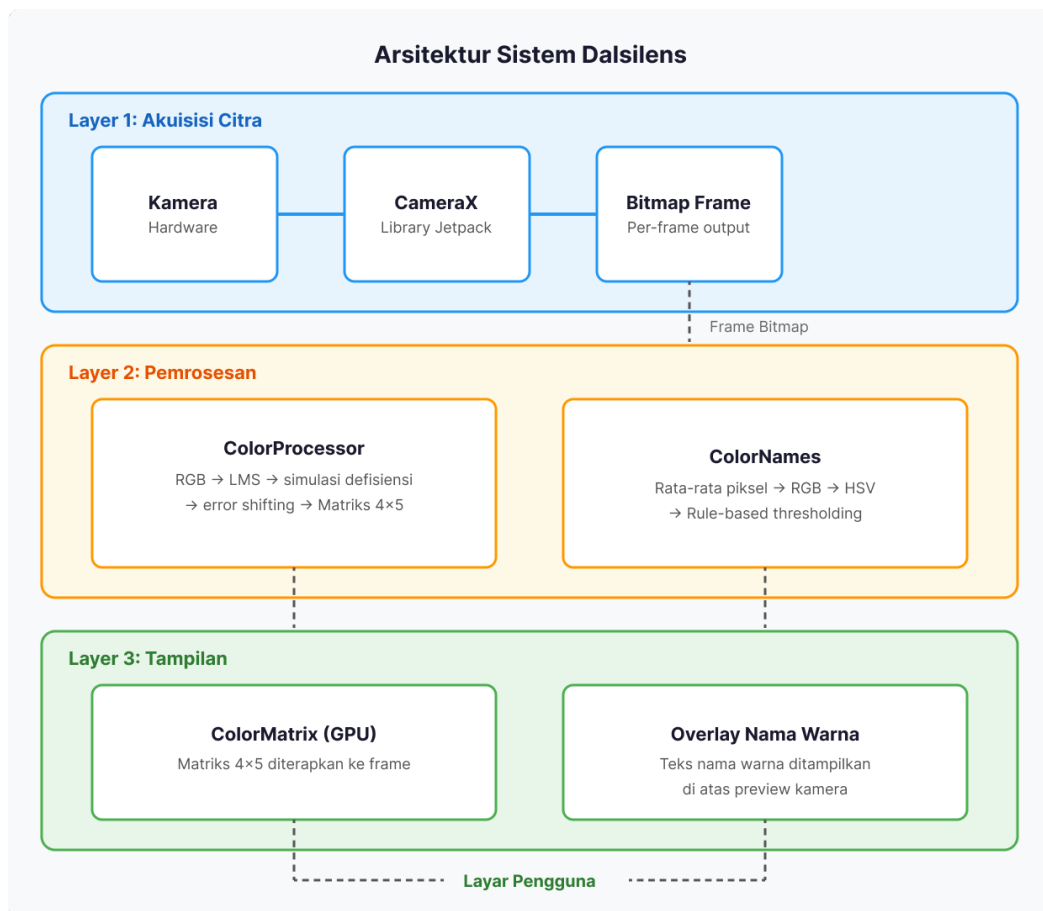
3.1. Gambaran Umum Sistem

Dalsilens adalah aplikasi Android yang dirancang sebagai alat bantu visual bagi penderita dikromasi. Aplikasi ini menggabungkan dua fitur utama dalam satu antarmuka berbasis kamera secara *real-time*, yaitu koreksi dan simulasi warna menggunakan algoritma *LMS Daltonization*, serta identifikasi nama warna berbasis ruang warna HSV. Secara umum, alur kerja sistem dimulai dari pengambilan citra oleh kamera, pemrosesan piksel, transformasi warna, hingga tampilan hasil kepada pengguna, semua berjalan dalam satu siklus per *frame* yang berulang terus-menerus selama aplikasi aktif. Gambar 3.1 menggambarkan hubungan antar komponen utama sistem Dalsilens.

Sistem terbagi menjadi tiga lapisan utama. Lapisan pertama adalah lapisan akuisisi, di mana *library* CameraX menangkap *frame* dari kamera secara berkelanjutan dan mengonversinya menjadi objek *Bitmap*. Lapisan kedua adalah lapisan pemrosesan, di mana *ColorProcessor* menghitung matriks transformasi warna berdasarkan jenis defisiensi dan mode yang dipilih pengguna, sementara *ColorNames* mengekstrak nilai rata-rata piksel dan mengklasifikasikan nama warna melalui *rule-based thresholding* pada ruang warna HSV. Lapisan ketiga adalah lapisan tampilan, di mana hasil transformasi diaplikasikan ke *frame* menggunakan *ColorMatrix* yang diakselerasi GPU, dan nama warna ditampilkan secara *overlay* di layar.

3.1.1. Metode Pengembangan Perangkat Lunak

Pengembangan aplikasi Dalsilens mengikuti model *Waterfall* sebagai metodologi *Software Development Life Cycle* (SDLC) yang dipilih. Model *Waterfall* dipilih karena kebutuhan sistem sudah terdefinisi dengan jelas sejak awal penelitian, jenis



Gambar 3.1: Gambaran Umum Arsitektur Sistem Dalsilens

defisiensi yang ditangani, platform target, algoritma yang digunakan, serta fitur-fitur yang harus ada. Sehingga tidak diperlukan iterasi perubahan kebutuhan di tengah proses pengembangan (IBM, 2026). Setiap fase pada model ini diselesaikan secara berurutan sebelum melanjutkan ke fase berikutnya, sebagaimana digambarkan pada Gambar 3.2.



Gambar 3.2: Tahapan Model *Waterfall* pada Pengembangan Dalsilens

Berikut adalah penerapan setiap fase *Waterfall* pada pengembangan aplikasi Dalsilens.

1. Analisis Kebutuhan (*Requirements Analysis*)

Pada fase ini dilakukan identifikasi masalah yang dihadapi penderita dikromasi serta analisis kebutuhan fungsional dan non-fungsional aplikasi. Kebutuhan fungsional mencakup fitur simulasi dan koreksi tiga jenis dikromasi (*Protanopia*, *Deutanopia*, *Tritanopia*), fitur identifikasi nama warna secara *real-time*, serta pengaturan *severity* dan ukuran area *sampling*. Kebutuhan non-fungsional mencakup performa *real-time* dengan latensi rendah dan kompatibilitas dengan perangkat Android.

2. Perancangan Sistem (*System Design*)

Fase ini menghasilkan rancangan arsitektur sistem dan diagram UML yang

menjadi acuan implementasi. Perancangan mencakup *Use Case Diagram* untuk mendefinisikan interaksi pengguna, *Activity Diagram* untuk menggambarkan alur proses akuisisi citra dan daltonisasi, serta *Class Diagram* untuk mendefinisikan struktur komponen perangkat lunak dan hubungan antar kelas.

3. Implementasi (*Implementation*)

Pada fase ini seluruh komponen sistem dikodekan menggunakan bahasa Kotlin di lingkungan Android Studio. Implementasi mencakup modul *ColorProcessor* untuk konversi ruang warna dan pembuatan matriks transformasi, modul *ColorNames* untuk klasifikasi nama warna berbasis HSV, serta integrasi *library* CameraX untuk akuisisi citra dan *ColorMatrix* untuk penerapan transformasi berbasis GPU.

4. Pengujian (*Testing*)

Fase pengujian dilakukan untuk memverifikasi bahwa seluruh fitur berjalan sesuai dengan kebutuhan yang telah ditentukan. Pengujian mencakup empat aspek yaitu pengujian fungsionalitas menggunakan metode *Black Box Testing*, pengujian akurasi klasifikasi nama warna, pengujian performa *real-time* menggunakan *Android Studio Profiler*, dan pengujian kebergunaan menggunakan metode *System Usability Scale* (SUS).

5. Pemeliharaan (*Maintenance*)

Meskipun aplikasi Dalsilens dikembangkan dalam konteks penelitian, fase pemeliharaan tetap diidentifikasi dalam bentuk saran pengembangan lanjutan. Potensi pengembangan mencakup peningkatan akurasi klasifikasi warna menggunakan pendekatan *machine learning*, penambahan dukungan tipe defisiensi *anomalous trichromacy*, serta pengembangan fitur aksesibilitas tambahan.

3.2. Arsitektur Sistem

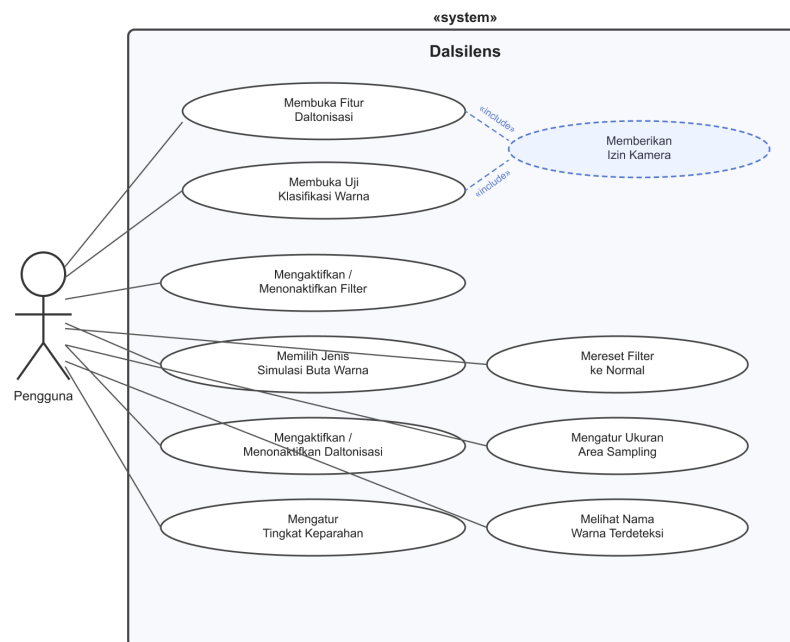
Secara keseluruhan, sistem Dalsilens dirancang untuk memproses gambar dari kamera secara *real-time* dengan latensi yang rendah. Supaya proses berjalan efisien,

komputasi dibagi menjadi dua tahap utama, yaitu pengambilan gambar untuk deteksi warna, dan manipulasi warna pada level *bitmap* menggunakan akselerasi GPU melalui *Color Matrix*.

3.2.1. Perancangan UML

3.2.1.1 Use Case Diagram

Use case diagram pada Gambar 3.3 menggambarkan interaksi antara pengguna dengan fitur-fitur yang tersedia pada aplikasi Dalsilens. Terdapat satu aktor dalam sistem ini, yaitu pengguna (penderita buta warna) yang berinteraksi langsung dengan aplikasi.



Gambar 3.3: *Use Case Diagram* Aplikasi Dalsilens

Berikut adalah penjelasan dari masing-masing *use case* yang tersedia:

1. Membuka Fitur Daltonisasi

Dari halaman *dashboard*, pengguna dapat masuk ke halaman kamera utama yang menjalankan fitur koreksi dan simulasi warna secara *real-time*. Sebelum

masuk, sistem mengecek izin kamera. Jika belum diberikan, aplikasi meminta izin terlebih dahulu.

2. Membuka Uji Klasifikasi Warna

Pengguna juga dapat membuka halaman uji klasifikasi warna (*Color Classification Test*) untuk menguji kemampuan sistem dalam mengenali nama warna dari tangkapan kamera.

3. Mengaktifkan/Menonaktifkan Filter

Pengguna dapat menghidupkan atau mematikan filter warna melalui tombol *toggle* di layar kamera. Saat filter dimatikan, gambar ditampilkan dalam kondisi asli tanpa efek apapun.

4. Memilih Jenis Simulasi Buta Warna

Pengguna dapat memilih jenis defisiensi yang ingin disimulasikan atau dikoreksi, yaitu *Normal*, *Protanopia*, *Deutanopia*, *Tritanopia*, atau *Custom*. Pilihan ini dapat diakses di panel pengaturan (*bottom sheet*).

5. Mengaktifkan/Menonaktifkan Daltonisasi

Pengguna dapat mengaktifkan mode daltonisasi melalui tombol khusus di layar atau *switch* di panel pengaturan. Saat aktif, sistem akan mensimulasikan kondisi buta warna dan menerapkan algoritma koreksi warna.

6. Mengatur Tingkat Keparahan (*Severity*)

Pengguna dapat menyesuaikan intensitas efek filter melalui *slider* di panel pengaturan, dengan rentang nilai 0,0 hingga 1,0.

7. Mengatur Ukuran Area Sampling

Pengguna dapat mengubah radius area pengambilan sampel piksel untuk deteksi warna, yang memengaruhi seberapa luas area di sekitar *crosshair* yang dianalisis.

8. Mereset Filter ke Normal

Pengguna dapat mereset semua pengaturan filter kembali ke kondisi *Normal*

(tanpa efek) menggunakan tombol reset di panel pengaturan.

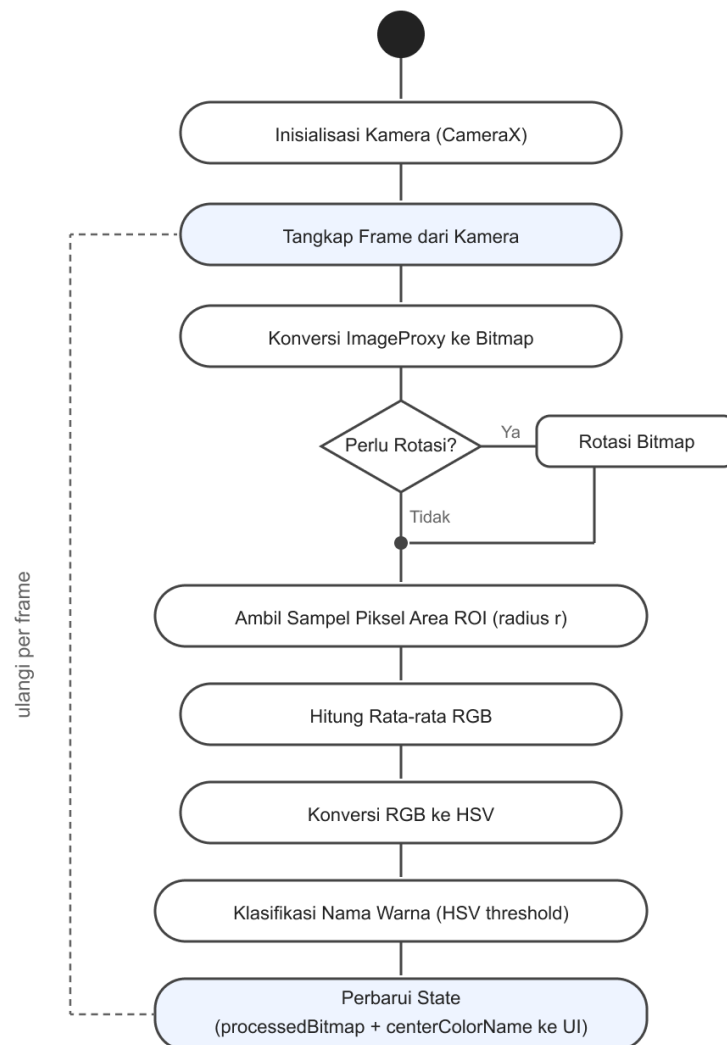
9. Melihat Nama Warna yang Terdeteksi

Sistem secara otomatis menampilkan nama warna dari objek yang dituju oleh *crosshair* di tengah layar secara *real-time*.

3.2.1.2 Activity Diagram

Activity diagram digunakan untuk menggambarkan alur aktivitas yang terjadi di dalam sistem Dalsilens. Karena sistem memiliki dua alur komputasi utama, masing-masing digambarkan dalam diagram terpisah.

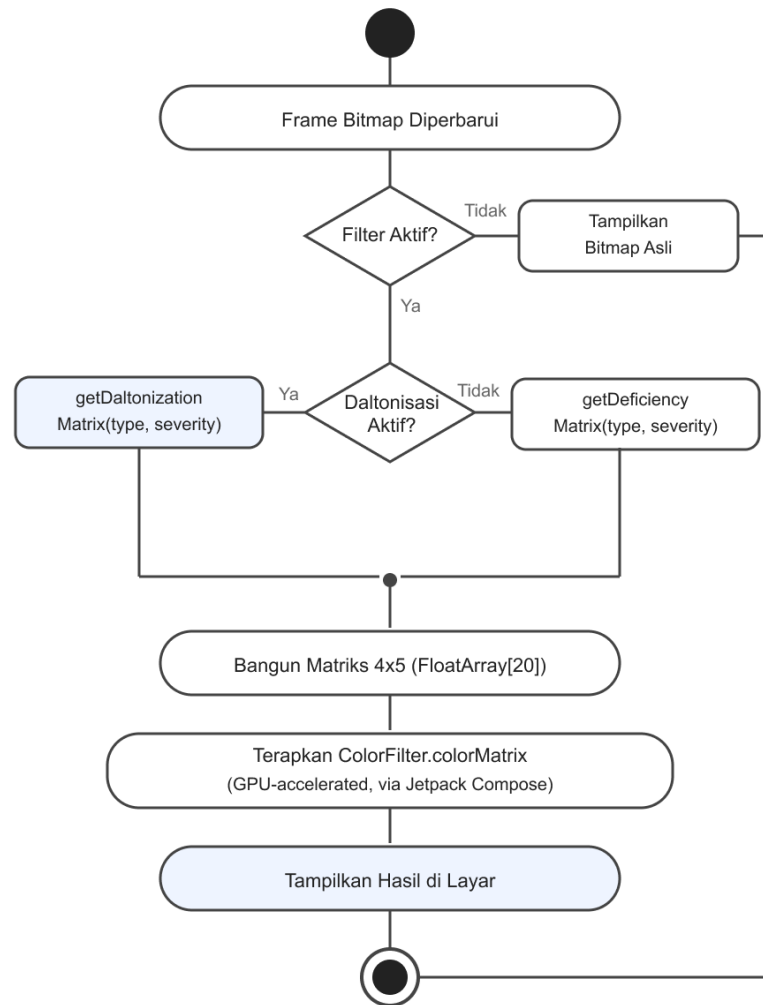
Gambar 3.4 menunjukkan alur aktivitas pengambilan gambar dan identifikasi warna. Saat pengguna membuka *CameraScreen*, sistem menginisialisasi kamera menggunakan *CameraX* dan mulai menangkap *frame* secara terus-menerus. Setiap *frame* diteruskan ke *ImageAnalysis*, dikonversi menjadi *bitmap*, lalu dirotasi jika diperlukan sesuai orientasi sensor. Sistem kemudian mengambil rata-rata nilai RGB dari piksel di area tengah layar menggunakan kelas *ColorNames*, mengonversinya ke ruang warna HSV, dan mengklasifikasikannya menjadi nama warna yang ditampilkan di layar.



Gambar 3.4: *Activity Diagram* Pengambilan Gambar dan Identifikasi Warna

Gambar 3.5 menunjukkan alur aktivitas proses daltonisasi. Setiap kali *bitmap* diperbarui, sistem mengecek apakah filter diaktifkan oleh pengguna. Jika tidak aktif, gambar ditampilkan apa adanya. Jika aktif, sistem mengecek apakah mode daltonisasi dinyalakan. Jika ya, `ColorProcessor.getDaltonizationMatrix()` dipanggil untuk menghasilkan matriks koreksi. Jika tidak, `ColorProcessor.getDeficiencyMatrix()` dipanggil untuk menghasilkan matriks simulasi. Kedua fungsi ini mempertimbangkan jenis defisiensi yang dipilih dan nilai *severity*. Matriks yang dihasilkan kemudian diterapkan ke gambar melalui *Color Matrix Filter*

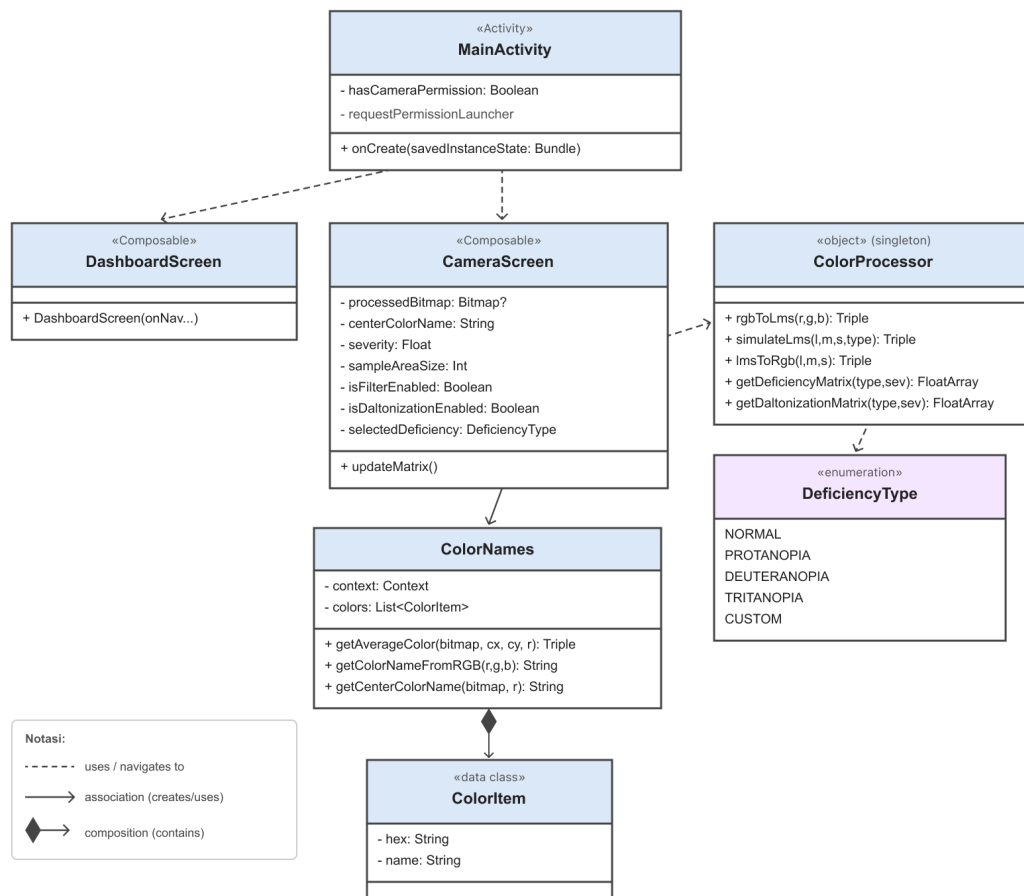
yang dirender oleh GPU.



Gambar 3.5: *Activity Diagram* Proses Daltonisasi

3.2.1.3 Class Diagram

Class diagram pada Gambar 3.6 menggambarkan struktur kelas-kelas utama yang menyusun aplikasi Dalsilens beserta hubungan antar kelasnya.



Gambar 3.6: *Class Diagram* Aplikasi Dalsilens

Berikut adalah penjelasan dari masing-masing kelas yang terlibat:

1. MainActivity

Kelas utama yang menjadi titik masuk aplikasi, mewarisi `ComponentActivity` dari Jetpack. Kelas ini bertanggung jawab memeriksa dan meminta izin kamera, serta mengatur navigasi antar layar melalui `NavHost`. `MainActivity` menginisialisasi dua layar utama, yaitu `DashboardScreen` dan `CameraScreen`.

2. DashboardScreen

Fungsi *composable* yang menampilkan halaman awal aplikasi. Berisi dua tombol navigasi, yang pertama untuk masuk ke halaman kamera (*fitur dalton-*

isasi) dan yang kedua untuk membuka halaman uji klasifikasi warna.

3. **CameraScreen**

Fungsi *composable* yang merupakan layar utama dan terkompleks dalam aplikasi. Kelas ini menyimpan seluruh *state* yang dibutuhkan tampilan, di antaranya `processedBitmap`, `centerColorName`, `severity`, `sampleAreaSize`, `isFilterEnabled`, `isDaltonizationEnabled`, dan `selectedDeficiency`. `CameraScreen` memanggil `ColorProcessor` untuk mendapatkan matriks transformasi, dan menggunakan `ColorNames` untuk mengidentifikasi nama warna secara *real-time*.

4. **ColorProcessor**

Kelas berbentuk *singleton* (object dalam Kotlin) yang memusatkan seluruh komputasi matematis terkait pemrosesan warna. Menyediakan fungsi konversi RGB ke LMS (`rgbToLms`), simulasi buta warna (`simulateLms`), konversi balik LMS ke RGB (`lmsToRgb`), serta pembangkitan matriks 4×5 untuk simulasi (`getDeficiencyMatrix`) dan daltonisasi (`getDaltonizationMatrix`).

5. **DeficiencyType**

Enumerasi yang mendefinisikan jenis-jenis defisiensi warna yang didukung oleh aplikasi, yaitu `NORMAL`, `PROTANOPIA`, `DEUTERANOPIA`, `TRITANOPIA`, dan `CUSTOM`. Nilai ini digunakan oleh `ColorProcessor` untuk menentukan matriks transformasi yang tepat.

6. **ColorNames**

Kelas yang menangani seluruh proses identifikasi nama warna. Menerima *bitmap* dari kamera, mengambil sampel piksel di sekitar koordinat tengah (`getAverageColor`), mengonversinya ke ruang warna HSV, lalu mengklasifikasikan hasilnya menjadi nama warna (`getColorNameFromRGB`). Kelas ini juga menyimpan daftar `ColorItem` yang dimuat dari berkas `color_names.json`.

7. **ColorItem**

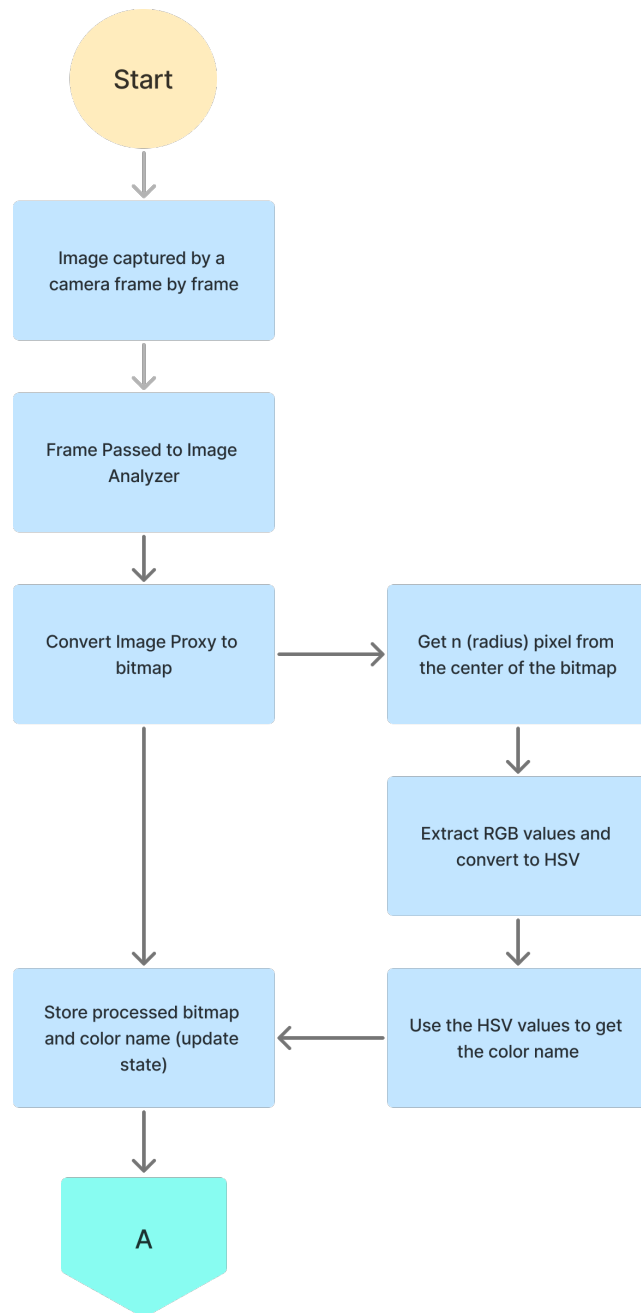
Data class sederhana yang merepresentasikan satu entri warna dengan dua atribut yaitu *hex* (kode warna heksadesimal) dan *name* (nama warna). Kelas ini digunakan oleh *ColorNames* sebagai elemen dari daftar referensi warna yang dimuat dari aset aplikasi.

3.2.2. Pengambilan Gambar dan Identifikasi Warna

Tahap pertama adalah pengambilan gambar dari kamera sekaligus mengidentifikasi warna objek secara *real-time*. Alur proses ini ditunjukkan pada Gambar 3.7.

Saat pengguna membuka aplikasi, sistem langsung menginisialisasi kamera menggunakan pustaka *CameraX*. Kamera kemudian menangkap *frame* secara terus-menerus dan mengirimkannya ke komponen *ImageAnalysis*. Di sini, gambar dikonversi menjadi *bitmap* agar bisa diproses lebih lanjut.

Setelah *bitmap* tersedia, sistem membuat sebuah area kecil di tengah layar yang disebut *Region of Interest* (ROI), yaitu area yang ditandai oleh *crosshair*. Piksel-piksel di dalam ROI diambil, dirata-ratakan, lalu hasilnya dikonversi ke ruang warna HSV untuk menentukan nama warna dominan. Hasilnya berupa *bitmap* dan nama warna tersebut disimpan sebagai *state*, yang akan memicu pembaruan tampilan secara otomatis.

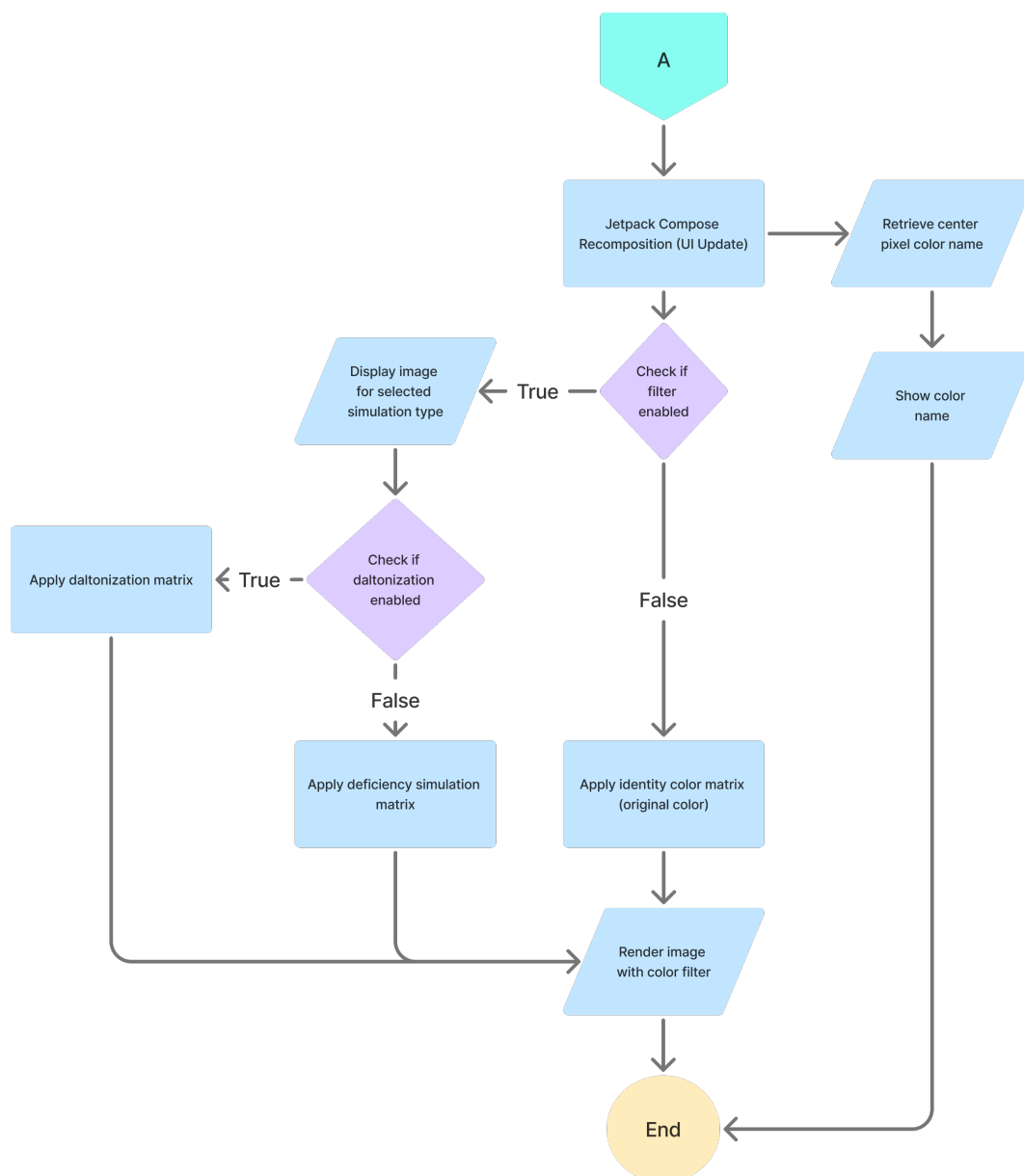


Gambar 3.7: Diagram Alur Pengambilan Gambar dan Identifikasi Warna

3.2.3. Proses Daltonisasi

Tahap kedua adalah proses daltonisasi yang berjalan setiap kali *bitmap* diperbarui. Alur lengkapnya dapat dilihat pada Gambar 3.8.

Sistem pertama-tama mengecek apakah filter warna diaktifkan atau tidak. Jika tidak aktif, sistem menerapkan *color matrix* identitas sehingga gambar tampil seperti aslinya. Jika filter aktif, sistem menerapkan matriks transformasi sesuai jenis buta warna yang dipilih (simulasi atau daltonisasi). Setelah matriks ditentukan, gambar dirender menggunakan *Color Matrix Filter* yang berjalan di GPU, sehingga proses ini efisien dan tidak membebani CPU.



Gambar 3.8: Diagram Alur Simulasi Buta Warna dan Proses Daltonisasi

3.3. Implementasi Algoritma *LMS Daltonization*

Algoritma *LMS Daltonization* bekerja dengan cara mengonversi warna dari ruang warna RGB ke ruang warna LMS, mensimulasikan kondisi buta warna, lalu menggeser informasi warna yang hilang ke spektrum yang masih bisa dilihat. Berikut adalah tahapan implementasinya.

3.3.1. Konversi Ruang Warna RGB ke LMS

Langkah pertama adalah mengonversi warna dari RGB ke LMS. Ruang warna LMS merepresentasikan cara kerja tiga jenis sel kerucut pada mata manusia, yaitu *Long* (L), *Medium* (M), dan *Short* (S) (Tasnim dan Hasan, 2017). Berbeda dengan RGB yang berfokus pada perangkat layar, LMS lebih mencerminkan persepsi penglihatan manusia secara biologis.

Konversi ini dilakukan dengan mengalikan nilai piksel RGB dengan matriks konversi berikut (Viénot, Brettel dan Mollon, 1999; Muhammad, Maitama, Bature dan Yunusa, 2020; Tasnim dan Hasan, 2017):

$$\begin{pmatrix} L \\ M \\ S \end{pmatrix} = \begin{pmatrix} 17,8824 & 43,5161 & 4,11935 \\ 3,4565 & 27,1554 & 3,86714 \\ 0,02996 & 0,18431 & 1,46709 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix} \quad (3.1)$$

Setiap nilai L, M, dan S merupakan kombinasi dari ketiga kanal RGB, yang mencerminkan seberapa besar kontribusi masing-masing warna terhadap respons sel kerucut.

3.3.2. Simulasi Buta Warna

Setelah nilai LMS diperoleh, langkah berikutnya adalah mensimulasikan kondisi buta warna. Simulasi dilakukan dengan memodifikasi salah satu komponen LMS sesuai jenis defisiensinya. Pada kondisi ini, komponen yang terpengaruh tidak digu-

nakan secara langsung, melainkan dihitung ulang dari dua komponen lainnya (Viénot, Brettel dan Mollon, 1999; Muhammad, Maitama, Bature dan Yunusa, 2020; Tasnim dan Hasan, 2017).

Berikut adalah matriks simulasi untuk masing-masing jenis buta warna:

Protanopia (buta warna merah)

$$\begin{pmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{pmatrix} = \begin{pmatrix} 0 & 2,02344 & -2,52581 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} L \\ M \\ S \end{pmatrix} \quad (3.2)$$

Deutanopia (buta warna hijau)

$$\begin{pmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0,494207 & 0 & 1,24827 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} L \\ M \\ S \end{pmatrix} \quad (3.3)$$

Tritanopia (buta warna biru)

$$\begin{pmatrix} L_{sim} \\ M_{sim} \\ S_{sim} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -0,395913 & 0,801109 & 0 \end{pmatrix} \begin{pmatrix} L \\ M \\ S \end{pmatrix} \quad (3.4)$$

Hasil dari tahap ini adalah nilai LMS yang merepresentasikan warna sebagaimana yang terlihat oleh penderita buta warna.

3.3.3. Proses Daltonisasi

Setelah simulasi dilakukan, langkah selanjutnya adalah menghitung informasi warna yang *hilang* dan menggesernya ke kanal yang masih bisa dilihat.

Pertama, nilai LMS hasil simulasi dikonversi kembali ke RGB menggunakan

matriks invers berikut (Viénot, Brettel dan Mollon, 1999; Muhammad, Maitama, Bature dan Yunusa, 2020; Tasnim dan Hasan, 2017):

$$\begin{pmatrix} R_{\text{sim}} \\ G_{\text{sim}} \\ B_{\text{sim}} \end{pmatrix} = \begin{pmatrix} 0,080944 & -0,130504 & 0,116721 \\ -0,0102485 & 0,0540194 & -0,113615 \\ -0,000365294 & -0,00412163 & 0,693513 \end{pmatrix} \begin{pmatrix} L_{\text{sim}} \\ M_{\text{sim}} \\ S_{\text{sim}} \end{pmatrix} \quad (3.5)$$

Kemudian nilai *error* dihitung, yaitu selisih antara warna asli dan warna hasil simulasi:

$$E_R = R - R_{\text{sim}}, \quad E_G = G - G_{\text{sim}}, \quad E_B = B - B_{\text{sim}} \quad (3.6)$$

Nilai *error* ini merepresentasikan informasi warna yang tidak bisa dilihat oleh penderita buta warna. Agar informasi ini tidak hilang begitu saja, nilainya digeser ke kanal warna lain yang masih bisa dilihat, menggunakan matriks penggeseran yang berbeda untuk setiap jenis buta warna.

Pada *protanopia*, kanal merah tidak digunakan sehingga *error*-nya dialihkan ke hijau dan biru (Tasnim dan Hasan, 2017):

$$\begin{pmatrix} R_{\text{shift}} \\ G_{\text{shift}} \\ B_{\text{shift}} \end{pmatrix} = \begin{pmatrix} 0,0 & 0,0 & 0,0 \\ 0,9 & 0,1 & 0,0 \\ 0,9 & 0,0 & 0,1 \end{pmatrix} \begin{pmatrix} E_R \\ E_G \\ E_B \end{pmatrix} \quad (3.7)$$

Pada *deutanopia*, kanal hijau yang bermasalah sehingga *error*-nya dialihkan ke merah dan biru (Muhammad, Maitama, Bature dan Yunusa, 2020):

$$\begin{pmatrix} R_{\text{shift}} \\ G_{\text{shift}} \\ B_{\text{shift}} \end{pmatrix} = \begin{pmatrix} 1,0 & 0,7 & 0,0 \\ 0,0 & 0,0 & 0,0 \\ 0,0 & 0,7 & 1,0 \end{pmatrix} \begin{pmatrix} E_R \\ E_G \\ E_B \end{pmatrix} \quad (3.8)$$

Pada *tritanopia*, kanal biru yang bermasalah sehingga *error*-nya dialihkan ke merah dan hijau (Schmitz, 2026):

$$\begin{pmatrix} R_{\text{shift}} \\ G_{\text{shift}} \\ B_{\text{shift}} \end{pmatrix} = \begin{pmatrix} 1,0 & 0,0 & 0,7 \\ 0,0 & 1,0 & 0,7 \\ 0,0 & 0,0 & 0,0 \end{pmatrix} \begin{pmatrix} E_R \\ E_G \\ E_B \end{pmatrix} \quad (3.9)$$

Terakhir, nilai *shift* ditambahkan ke warna piksel asli untuk menghasilkan warna yang telah dikoreksi:

$$R_{\text{dalton}} = R + R_{\text{shift}}, \quad G_{\text{dalton}} = G + G_{\text{shift}}, \quad B_{\text{dalton}} = B + B_{\text{shift}} \quad (3.10)$$

3.3.4. Konstruksi Matriks Transformasi 4×5

Agar proses daltonisasi tidak perlu menghitung ulang setiap piksel satu per satu, seluruh transformasi pada persamaan 3.10 diringkas menjadi sebuah matriks konstan berukuran 4×5 .

Caranya adalah dengan mengevaluasi fungsi transformasi T terhadap tiga warna dasar, yaitu merah murni $(1, 0, 0)$, hijau murni $(0, 1, 0)$, dan biru murni $(0, 0, 1)$:

$$T(1, 0, 0) = (r_R, r_G, r_B), \quad T(0, 1, 0) = (g_R, g_G, g_B), \quad T(0, 0, 1) = (b_R, b_G, b_B) \quad (3.11)$$

Hasil dari ketiga evaluasi ini disusun menjadi matriks M_{dalton} berikut, yang sekaligus mempertahankan nilai *alpha* (transparansi) piksel di baris keempat:

$$M_{\text{dalton}} = \begin{pmatrix} r_R & g_R & b_R & 0 & 0 \\ r_G & g_G & b_G & 0 & 0 \\ r_B & g_B & b_B & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} \quad (3.12)$$

Matriks ini kemudian langsung dipakai oleh *Color Matrix Filter* di GPU, sehingga koreksi warna bisa dirender pada setiap *frame* secara efisien.

3.4. Implementasi Tingkat Keparahan (*Severity Level*)

Fitur *severity level* memungkinkan pengguna mengatur seberapa kuat efek filter yang diterapkan, mulai dari 0,0 (tidak ada efek) hingga 1,0 (efek penuh). Karena tujuannya berbeda, cara penerapan *severity* dibedakan antara mode simulasi dan mode daltonisasi.

3.4.1. Keparahan pada Mode Simulasi

Pada mode simulasi, *severity* berfungsi untuk mengatur intensitas efek simulasi buta warna yang ditampilkan, dari kondisi penglihatan normal hingga dikromasi penuh. Perhitungan dilakukan langsung di ruang warna LMS menggunakan interpolasi linear antara nilai LMS penglihatan normal dan nilai LMS defisiensi penuh, yang secara konseptual terinspirasi dari pendekatan interpolasi bertahap pada penelitian Machado, Oliveira dan Fernandes (2009). Dalam penelitian ini, interpolasi ini merupakan pendekatan sederhana (*approximation*) untuk mengatur intensitas filter dan belum merepresentasikan model *Anomalous Trichromacy* secara khusus, karena kondisi tersebut melibatkan perubahan kurva sensitivitas sel kerucut, bukan sekadar interpolasi linear menuju dikromasi penuh.

$$L_{\text{final}} = L_{\text{normal}} + (L_{\text{sim}} - L_{\text{normal}}) \times S \quad (3.13)$$

$$M_{\text{final}} = M_{\text{normal}} + (M_{\text{sim}} - M_{\text{normal}}) \times S \quad (3.14)$$

$$S_{\text{final}} = S_{\text{normal}} + (S_{\text{sim}} - S_{\text{normal}}) \times S \quad (3.15)$$

Ketika $S = 0$, hasil simulasi sama dengan penglihatan normal. Ketika $S = 1$, simulasi menampilkan kondisi defisiensi penuh.

3.4.2. Keparahan pada Mode Daltonisasi

Pada mode daltonisasi, *severity* berfungsi untuk mengatur seberapa kuat kompensasi warna yang diberikan. Perhitungan dilakukan di ruang warna RGB dengan cara mengalikan nilai *error shift* setiap kanal dengan nilai *severity* sebelum ditambahkan ke piksel asli:

$$R_{\text{dalton}} = R_{\text{asli}} + (\text{Shift}_R \times S) \quad (3.16)$$

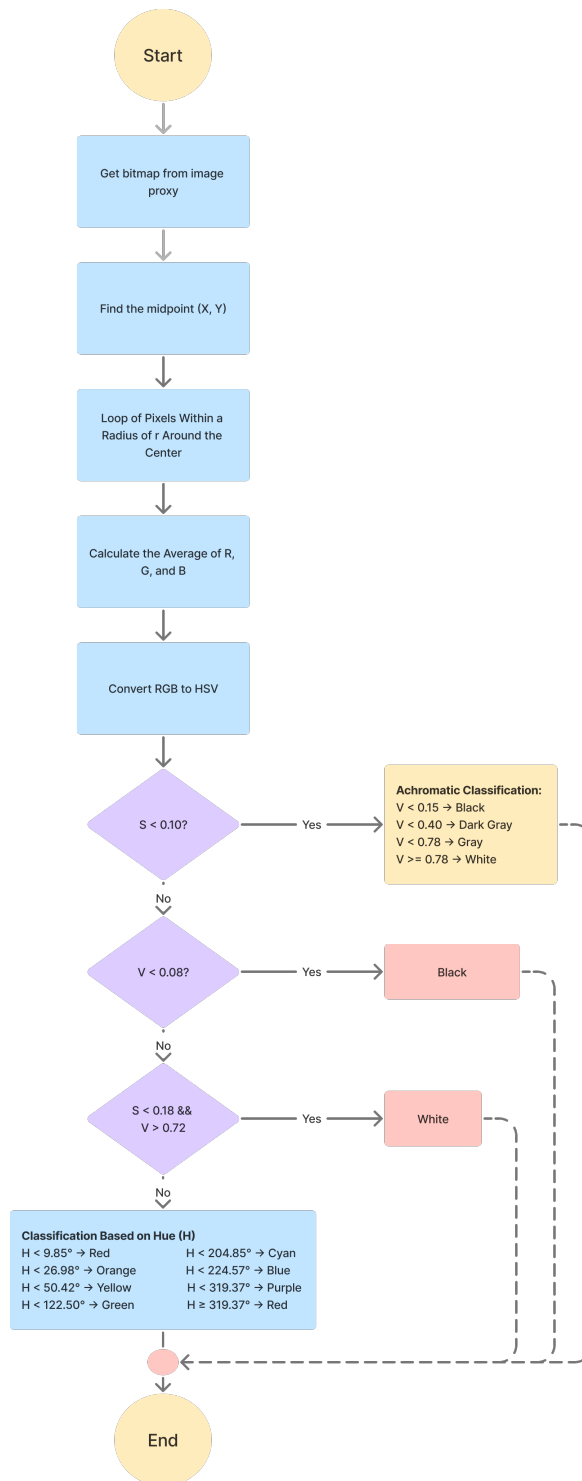
$$G_{\text{dalton}} = G_{\text{asli}} + (\text{Shift}_G \times S) \quad (3.17)$$

$$B_{\text{dalton}} = B_{\text{asli}} + (\text{Shift}_B \times S) \quad (3.18)$$

Dengan cara ini, pengguna bisa menyesuaikan intensitas koreksi warna sesuai kebutuhan dan kenyamanan visual mereka.

3.5. Ekstraksi Warna Dominan Berbasis HSV

Fitur ini bertujuan untuk mendeteksi dan menampilkan nama warna dari objek yang dituju oleh *crosshair* secara *real-time*. Alur prosesnya ditunjukkan pada Gambar 3.9.



Gambar 3.9: Diagram Alur Ekstraksi dan Klasifikasi Warna Secara *Real-Time*

3.5.1. Pengambilan *Bitmap* dari Kamera

Setiap *frame* dari kamera diambil oleh komponen *ImageAnalysis* pada *CameraX*, lalu dikonversi menjadi *Bitmap*. Karena orientasi sensor kamera bisa berbeda-beda antar perangkat Android, nilai rotasi dari metadata *frame* juga diperiksa. Jika rotasinya tidak nol, *bitmap* dirotasi terlebih dahulu sebelum diproses.

3.5.2. Pengambilan Sampel Piksel

Warna tidak diambil dari satu piksel saja, karena satu piksel rentan terhadap *noise* dan variasi cahaya. Sebagai gantinya, sistem mengambil sampel dari area kotak di sekitar titik tengah layar.

Radius area sampling yang digunakan secara *default* adalah $r = 5$, sehingga area yang disampling berukuran $11 \times 11 = 121$ piksel, yang dihitung dengan persamaan:

$$N = (2r + 1) \times (2r + 1) \quad (3.19)$$

Agar koordinat piksel tidak keluar dari batas gambar, digunakan fungsi pembatas berikut:

$$f(x) = \min(\max(x, 0), W - 1), \quad g(y) = \min(\max(y, 0), H - 1) \quad (3.20)$$

Setiap piksel disimpan sebagai bilangan bulat 32-bit dalam format ARGB. Komponen warna diekstrak dengan operasi *bit shift* dan *masking*:

$$B_{(x,y)} = \text{pixel} \& 255, \quad G_{(x,y)} = (\text{pixel} \gg 8) \& 255, \quad R_{(x,y)} = (\text{pixel} \gg 16) \& 255 \quad (3.21)$$

Nilai rata-rata RGB dari seluruh piksel dalam area sampling kemudian dihitung:

$$\bar{R} = \frac{1}{N} \sum_{i=1}^N R_i, \quad \bar{G} = \frac{1}{N} \sum_{i=1}^N G_i, \quad \bar{B} = \frac{1}{N} \sum_{i=1}^N B_i \quad (3.22)$$

3.5.3. Klasifikasi Nama Warna

Nilai rata-rata RGB tersebut kemudian dikonversi ke ruang warna HSV menggunakan fungsi bawaan Android `RGBToHSV()`. Hasilnya adalah tiga komponen, yaitu *Hue* (H) sebagai jenis warna (0–360), *Saturation* (S) sebagai intensitas warna (0–1), dan *Value* (V) sebagai kecerahan (0–1).

Nama warna ditentukan berdasarkan rentang nilai H, S, dan V menggunakan pendekatan *threshold*. Nilai *threshold* ini ditentukan melalui pengujian empiris pada dua kondisi pencahayaan yaitu normal dan redup. Sebagai contoh, untuk warna merah, rata-rata Hue pada kondisi normal adalah 3,3 dan pada kondisi redup adalah 8,1. Standar deviasi dihitung menggunakan:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2} \quad (3.23)$$

Rentang toleransi ditentukan dengan:

$$\text{Rentang Toleransi} = \bar{H} \pm 2\sigma \quad (3.24)$$

Dari hasil perhitungan, batas atas untuk warna merah ditetapkan pada 9,86 (mengambil batas kondisi terburuk). Proses yang sama diterapkan untuk semua warna lainnya, menghasilkan tabel *threshold* pada Tabel 3.1 dan Tabel 3.2.

Tabel 3.1: Klasifikasi Warna Berdasarkan Rentang *Hue*

Rentang <i>Hue</i> (°)	Nama Warna
$H < 9,85$	Merah
$9,85 \leq H < 26,98$	Oranye
$26,98 \leq H < 50,42$	Kuning
$50,42 \leq H < 122,50$	Hijau
$122,50 \leq H < 204,85$	Cyan
$204,85 \leq H < 224,57$	Biru
$224,57 \leq H < 319,37$	Ungu
$H \geq 319,37$	Merah

Apabila nilai *Saturation* rendah (warna tidak jenuh), klasifikasi tidak lagi menggunakan *Hue* melainkan menggunakan *Value* untuk menentukan apakah warna tersebut hitam, abu-abu, atau putih, seperti pada Tabel 3.2.

Tabel 3.2: Klasifikasi Warna Netral Berdasarkan *Saturation* dan *Value*

Kondisi	Rentang <i>Value</i>	Nama Warna
$S < 0,10$	$V < 0,15$	Hitam
$S < 0,10$	$0,15 \leq V < 0,40$	Abu Gelap
$S < 0,10$	$0,40 \leq V < 0,78$	Abu-Abu
$S < 0,10$	$V \geq 0,78$	Putih

Meskipun nilai *threshold* seperti $S < 0,10$ dan rentang *Value* ditentukan berdasarkan hasil pengujian langsung, pengelompokannya tetap berdasarkan teori ruang warna HSV. Ruang warna HSV memisahkan informasi warna dan intensitas cahaya, berbeda dengan RGB yang menggabungkan keduanya dalam tiga komponen sekaligus (Tasnim dan Hasan, 2017). Dengan pemisahan ini, parameter *Saturation* (S) dan *Value* (V) dapat dianalisis secara terpisah untuk mendeteksi warna netral tanpa harus mempertimbangkan *Hue* (H).

Pendekatan ini didasarkan pada karakteristik masing-masing komponen HSV. *Saturation* menunjukkan tingkat kemurnian warna, di mana nilai S yang mendekati nol menandakan warna kehilangan pigmennya dan cenderung berubah menjadi abu-abu atau putih. Oleh karena itu, kondisi $S < 0,10$ digunakan sebagai indikator awal untuk mendeteksi warna akromatik (Y. Sirisathitkul, Dinmeung, Noonsuk dan C. Sirisathitkul, 2025). Setelah warna teridentifikasi sebagai akromatik, komponen *Value* digunakan untuk membedakan tingkat gelap dan terang, karena V secara langsung menunjukkan tingkat kecerahan piksel, mulai dari hitam ($V = 0$) hingga putih atau warna paling terang ($V = 1$) (Y. Sirisathitkul, Dinmeung, Noonsuk dan C. Sirisathitkul, 2025).

3.6. Skenario Pengujian

Pengujian dilakukan untuk memastikan aplikasi Dalsilens berjalan sesuai tujuan penelitian. Ada tiga skenario pengujian yang digunakan.

3.6.1. Pengujian Akurasi Klasifikasi Nama Warna

Pengujian akurasi dilakukan untuk mengevaluasi kemampuan fitur identifikasi nama warna dalam mengenali warna objek nyata secara *real-time*. Pengujian menggunakan objek fisik berwarna yang mewakili setiap kategori warna yang didukung aplikasi, yaitu merah, oranye, kuning, hijau, cyan, biru, ungu, putih, hitam, dan abu-abu. Setiap objek diuji dalam dua kondisi pencahayaan yaitu **normal** (cahaya ruangan standar) dan **redup** (cahaya dikurangi menggunakan tirai). Hasil deteksi nama warna yang ditampilkan aplikasi kemudian dibandingkan dengan warna objek yang sebenarnya. Deteksi dinyatakan **Benar** apabila nama warna yang ditampilkan sesuai dengan kategori warna objek uji, dan **Salah** apabila tidak sesuai. Akurasi dihitung menggunakan persamaan berikut:

$$\text{Akurasi} = \frac{\text{Jumlah Deteksi Benar}}{\text{Total Pengujian}} \times 100\% \quad (3.25)$$

Hasil pengujian pada setiap kondisi pencahayaan disajikan pada Tabel 3.3 dan Tabel 3.4.

Tabel 3.3: Hasil Pengujian Klasifikasi Warna pada Pencahayaan Normal

No.	Objek Uji	Warna Sebenarnya	Hasil Deteksi	Ket.
1	<i>Tali</i>	Merah	<i>Merah</i>	<i>Benar</i>
2	<i>Baju</i>	Oranye	<i>Oranye</i>	<i>Benar</i>
3	<i>Baju</i>	Kuning	<i>Kuning</i>	<i>Benar</i>
4	<i>Teko</i>	Hijau	<i>Hijau]</i>	<i>Benar</i>
5	<i>Pembersih Keyboard</i>	Cyan	<i>Cyan</i>	<i>Benar</i>
6	<i>Gantungan Kunci</i>	Biru	<i>Biru</i>	<i>Benar</i>
7	<i>KTM</i>	Ungu	<i>Magneta</i>	<i>Salah</i>
8	<i>Tumbler</i>	Putih	<i>Putih</i>	<i>Benar</i>
9	<i>Celana</i>	Hitam	<i>Hitam</i>	<i>Benar</i>
10	<i>Gagang jemuran</i>	Abu-abu	<i>Abu-abu</i>	<i>Benar</i>

Tabel 3.4: Hasil Pengujian Klasifikasi Warna pada Pencahayaan Redup

No.	Objek Uji	Warna Sebenarnya	Hasil Deteksi	Ket.
1	<i>Tali</i>	Merah	<i>Merah</i>	<i>Benar</i>
2	<i>Baju</i>	Oranye	<i>Oranye</i>	<i>Benar</i>
3	<i>Baju</i>	Kuning	<i>Oranye</i>	<i>Salah</i>
4	<i>Teko</i>	Hijau	<i>Hijau</i>	<i>Benar</i>

(lanjutan)

No.	Objek Uji	Warna Sebenarnya	Hasil Deteksi	Ket.
5	<i>Pembersih Keyboard</i>	Cyan	<i>Biru</i>	<i>Salah</i>
6	<i>Gantungan Kunci</i>	Biru	<i>Biru</i>	<i>Benar</i>
7	<i>KTM</i>	Ungu	<i>Merah</i>	<i>Salah</i>
8	<i>Tumbler</i>	Putih	<i>Abu-abu</i>	<i>Salah</i>
9	<i>Celana</i>	Hitam	<i>Hitam</i>	<i>Benar</i>
10	<i>Gagang jemuran</i>	Abu-abu	<i>Kuning</i>	<i>Salah</i>

Berdasarkan hasil pengujian, akurasi deteksi pada kondisi pencahayaan normal sebesar $\frac{9}{10} \times 100\% = 90\%$, dan pada kondisi pencahayaan redup sebesar $\frac{5}{10} \times 100\% = 50\%$. Penurunan akurasi pada kondisi redup terjadi karena nilai *Value* (V) pada ruang warna HSV menurun secara signifikan sehingga piksel yang seharusnya terklasifikasi berwarna cenderung terdeteksi sebagai abu-abu atau hitam.

3.6.2. Pengujian Performa *Real-Time*

Skenario ini memvalidasi efisiensi komputasi yang dioptimalkan oleh akselerasi GPU. Performa diukur berdasarkan hasil pengukuran dengan menghitung rata-rata nilai *Frame Per Second* (FPS) dan latensi pemrosesan saat sistem menjalankan transformasi *color matrix* dan klasifikasi warna secara bersamaan. Selain itu, penggunaan memori (RAM) dan utilisasi CPU juga dipantau menggunakan *Android Studio Profiler* untuk memastikan tidak ada *lag* atau *memory leak* selama aplikasi berjalan.

3.6.2.1 Pengujian FPS dan Latensi Pemrosesan

Pengujian FPS dan latensi dilakukan dengan menjalankan aplikasi pada mode Daltonisasi dengan fitur identifikasi nama warna aktif secara bersamaan, merepresentasikan kondisi beban komputasi tertinggi (*worst-case scenario*) yang mungkin terjadi pada aplikasi. Data statistik *rendering frame* diambil menggunakan perintah `adb shell dumpsys gfxinfo` pada setiap iterasi pengujian, yang menghasilkan distribusi (histogram) waktu *render* setiap *frame*. Nilai rata-rata latensi per iterasi dihitung dari rata-rata terbobot (*weighted average*) histogram tersebut, sedangkan nilai FPS diperoleh melalui konversi:

$$\text{FPS} = \frac{1000}{\text{Latensi Rata-rata (ms)}} \quad (3.26)$$

Pengujian dilakukan sebanyak tiga kali iterasi, masing-masing berjalan kurang lebih satu menit, untuk memperoleh gambaran konsistensi performa aplikasi selama digunakan secara terus-menerus. Hasil pengujian pada setiap iterasi ditunjukkan pada Tabel 3.5.

Tabel 3.5: Hasil Pengujian FPS dan Latensi Pemrosesan

Uji Coba	Total <i>Frame</i>	FPS	Latensi Rata-rata (ms)	<i>Janky Frames</i> (%)
Iterasi ke-1	2.389	55,0	18,2	28,59%
Iterasi ke-2	2.808	65,3	15,3	9,62%
Iterasi ke-3	1.134	46,8	21,4	23,10%
Rata-rata	6.331	57,2	17,5	19,19%

Berdasarkan Tabel 3.5, aplikasi Dalsilens memperoleh rata-rata performa sebesar 57,2 FPS dengan latensi pemrosesan rata-rata 17,5 ms per *frame* saat transformasi *color matrix* dan klasifikasi warna berjalan bersamaan. Nilai ini menunjukkan bahwa aplikasi mampu mempertahankan *frame rate* yang mendekati kondisi *real-time* yang mulus (60 FPS) meskipun kedua proses komputasi berjalan secara bersamaan.

Persentase *Janky Frames* rata-rata sebesar 19,19% menunjukkan bahwa sebagian kecil *frame* masih melewati batas waktu *render* ideal (16,67 ms), namun tidak sampai menyebabkan penurunan performa yang signifikan atau terasa oleh pengguna.

Data tambahan dari *GPU Rendering Profile* menunjukkan bahwa median waktu pemrosesan pada GPU berada pada kisaran 5–7 ms, jauh lebih rendah dibandingkan total waktu *render* per *frame*. Hal ini mengindikasikan bahwa beban komputasi transformasi warna melalui *ColorMatrix* yang diakselerasi GPU bukan merupakan titik hambat (*bottleneck*) utama, sehingga selaras dengan tujuan penelitian untuk memindahkan beban komputasi berat dari CPU ke GPU.

3.6.2.2 Pengujian Penggunaan Memori (RAM)

Pengujian penggunaan memori dilakukan pada skenario yang sama dengan pengujian FPS, yaitu saat mode Daltonisasi dan fitur identifikasi nama warna berjalan secara bersamaan. Data memori diambil menggunakan perintah `adb shell dumpsys meminfo` pada tiga iterasi pengujian, dengan mencatat nilai *Total Proportional Set Size* (PSS) sebagai representasi total pemakaian RAM aplikasi. *Proportional Set Size* (PSS) merupakan metode perhitungan memori pada Android yang membagi memori bersama (*shared memory*), seperti *library* sistem, secara proporsional ke setiap proses yang menggunakannya, sehingga nilai yang dihasilkan lebih akurat dan tidak tumpang tindih (*double-counting*) dibandingkan metode *Resident Set Size* (RSS) biasa. Pengujian ini bertujuan untuk memastikan tidak terjadi *memory leak*, yaitu kondisi di mana penggunaan memori terus meningkat secara konsisten tanpa pernah menurun akibat objek yang seharusnya dibersihkan (*garbage collected*) namun masih tertahan di memori. Hasil pengujian pada setiap iterasi ditunjukkan pada Tabel 3.6.

Tabel 3.6: Hasil Pengujian Penggunaan Memori (RAM)

Uji Coba	Total PSS (KB)	Total PSS (MB)
Iterasi ke-1	223.147	217,92
Iterasi ke-2	227.867	222,53
Iterasi ke-3	220.339	215,18
Rata-rata	223.784	218,54

Berdasarkan Tabel 3.6, aplikasi Dalsilens menggunakan rata-rata memori sebesar 218,54 MB selama pengujian. Nilai PSS pada ketiga pengujian berada dalam rentang yang sempit ($\pm 3,5$ MB dari rata-rata) dan tidak menunjukkan tren kenaikan yang konsisten dari iterasi pertama hingga ketiga. Apabila terjadi *memory leak*, nilai PSS semestinya terus meningkat secara bertahap pada setiap iterasi karena adanya objek yang seharusnya dibersihkan (*garbage collected*) namun masih tertahan di memori. Karena pola tersebut tidak ditemukan, dapat disimpulkan bahwa tidak ada indikasi *memory leak* pada aplikasi Dalsilens selama pengujian berlangsung.

3.6.2.3 Pengujian Utilisasi CPU

Pengujian utilisasi CPU dilakukan pada skenario yang sama dengan pengujian FPS dan RAM, yaitu saat mode Daltonisasi dan fitur identifikasi nama warna berjalan secara bersamaan. Data diambil menggunakan perintah `adb shell top -n 2 -d 1`, yang mengambil dua sampel dengan jeda satu detik agar persentase CPU dihitung berdasarkan selisih (*delta*) penggunaan yang valid, bukan hanya berdasarkan satu sampel pengukuran. Nilai yang dicatat pada setiap iterasi adalah sampel kedua dari setiap pasangan pengukuran. Hasil pengujian pada setiap iterasi ditunjukkan pada Tabel 3.7.

Tabel 3.7: Hasil Pengujian Utilisasi CPU

Uji Coba	Utilisasi CPU (%)
Iterasi ke-1	147%
Iterasi ke-2	148%
Iterasi ke-3	166%
Rata-rata	153,67%

Nilai utilisasi CPU pada Tabel 3.7 melebihi 100%. Hal ini bukan merupakan kesalahan pengukuran, melainkan konvensi umum pada utilitas sistem pemantauan berbasis Linux/Android seperti *top*, yang menghitung persentase CPU sebagai akumulasi penggunaan pada seluruh *core* yang dipakai oleh proses, bukan relatif terhadap satu *core* saja. Dengan demikian, nilai maksimum yang dapat dicapai suatu proses adalah $100\% \times N$, dengan N merupakan jumlah *core* CPU yang dimiliki perangkat. Berdasarkan informasi sistem pada perangkat uji, kapasitas total CPU tercatat sebesar 800%, yang menunjukkan bahwa perangkat memiliki 8 *core* ($N = 8$).

Apabila nilai rata-rata utilisasi CPU sebesar 153,67% dikonversi menjadi persentase terhadap kapasitas total perangkat, diperoleh:

$$\text{Utilisasi CPU Relatif} = \frac{153,67\%}{8 \times 100\%} \times 100\% \approx 19,21\% \quad (3.27)$$

Nilai ini menunjukkan bahwa aplikasi Dalsilens hanya menggunakan sekitar 19,21% dari total kapasitas komputasi CPU perangkat meskipun menjalankan transformasi *color matrix* dan klasifikasi warna secara bersamaan, sehingga proses tersebut tidak membebani CPU secara signifikan. Hal ini turut membuktikan efektivitas pemindahan beban komputasi berat dari CPU ke GPU melalui *Color Matrix* yang diakselerasi *hardware*, sebagaimana dijelaskan pada Subbab 2.4.2.

3.6.3. Pengujian Fungsionalitas (*Black Box Testing*)

Pengujian fungsionalitas dilakukan menggunakan metode *Black Box Testing*, yaitu pengujian yang berfokus pada perilaku sistem dari sudut pandang pengguna tanpa memperhatikan detail implementasi internal (Dicoding Indonesia, 2024). Setiap fitur diuji dengan memberikan masukan tertentu dan memverifikasi apakah keluaran yang dihasilkan sesuai dengan yang diharapkan. Hasil pengujian dicatat sebagai **Berhasil** jika keluaran sesuai ekspektasi, dan **Gagal** jika tidak.

Tabel 3.8: Hasil Pengujian *Black Box Testing* Aplikasi Dalsilens

No.	Skenario	Masukan	Ekspektasi	Hasil
1	Membuka aplikasi	Aplikasi dijalankan	Halaman <i>dashboard</i> tampil dengan dua tombol navigasi	Berhasil
2	Membuka fitur kamera saat izin belum diberikan	Tap tombol “Go to Camera”	Sistem meminta izin kamera kepada pengguna	Berhasil
3	Membuka fitur kamera saat izin sudah diberikan	Tap tombol “Go to Camera”	Aplikasi berpindah ke <i>CameraScreen</i> dan kamera aktif	Berhasil
4	Membuka fitur uji klasifikasi warna	Tap tombol “Color Classification Test”	Aplikasi berpindah ke halaman uji klasifikasi warna	Berhasil
5	Mengaktifkan filter warna	Tap tombol <i>toggle</i> filter	Filter aktif, tampilan kamera berubah sesuai jenis defisiensi yang dipilih	Berhasil

(lanjutan)

No.	Skenario	Masukan	Ekspektasi	Hasil
6	Menonaktifkan filter warna	Tap tombol <i>toggle</i> filter saat aktif	Filter nonaktif, tampilan kamera kembali ke kondisi asli	Berhasil
7	Mengaktifkan mode daltonisasi	Tap <i>switch</i> daltonisasi	Mode beralih dari simulasi ke koreksi warna (daltonisasi)	Berhasil
8	Memilih jenis simulasi <i>Protanopia</i>	Pilih “Protanopia” di panel pengaturan	Filter berubah mensimulasikan kondisi buta warna merah	Berhasil
9	Memilih jenis simulasi <i>Deutanopia</i>	Pilih “Deutanopia” di panel pengaturan	Filter berubah mensimulasikan kondisi buta warna hijau	Berhasil
10	Memilih jenis simulasi <i>Tritanopia</i>	Pilih “Tritanopia” di panel pengaturan	Filter berubah mensimulasikan kondisi buta warna biru	Berhasil
11	Mengatur <i>severity</i> ke nilai maksimum	Geser <i>slider severity</i> ke 1,0	Intensitas efek filter mencapai maksimum	Berhasil
12	Mengatur <i>severity</i> ke nilai minimum	Geser <i>slider severity</i> ke 0,0	Intensitas efek filter menjadi nol (tidak ada efek)	Berhasil

(lanjutan)

No.	Skenario	Masukan	Ekspektasi	Hasil
13	Mereset filter ke kondisi normal	Tap tombol reset di panel pengaturan	Semua pengaturan kembali ke <i>Normal</i> tanpa efek	Berhasil
14	Mengubah ukuran area <i>sampling</i>	Ubah nilai radius <i>sampling</i> di panel pengaturan	Area piksel yang disampling berubah sesuai nilai radius baru	Berhasil
15	Deteksi nama warna secara <i>real-time</i>	Arahkan kamera ke objek berwarna	Nama warna ditampilkan di layar secara otomatis dan berubah mengikuti objek	Berhasil

3.6.4. Pengujian Kebergunaan (*System Usability Scale*)

System Usability Scale (SUS) adalah kuesioner standar yang dikembangkan oleh Brooke (Brooke, 1995) untuk mengukur tingkat kebergunaan (*usability*) suatu sistem secara cepat dan efisien. Kuesioner SUS terdiri dari 10 pernyataan dengan skala *Likert* 1 hingga 5, di mana 1 berarti *Sangat Tidak Setuju* dan 5 berarti *Sangat Setuju*. Pernyataan bernomor ganjil bersifat positif, sedangkan pernyataan bernomor genap bersifat negatif (Lewis dan Sauro, 2017).

Skor SUS dihitung menggunakan persamaan berikut (Brooke, 1995):

$$\text{Skor SUS} = \left(\sum_{i \in \text{ganjil}} (x_i - 1) + \sum_{j \in \text{genap}} (5 - x_j) \right) \times 2,5 \quad (3.28)$$

di mana x_i adalah nilai jawaban responden untuk pernyataan ke- i . Hasil skor berada pada rentang 0 hingga 100. Interpretasi skor SUS mengacu pada klasifikasi yang

ditunjukkan pada Tabel 3.9.

Tabel 3.9: Klasifikasi Skor *System Usability Scale*

Rentang Skor	Nilai	Kategori
$> 80,3$	A	<i>Excellent</i>
$68 - 80,3$	B	<i>Good</i>
$57 - 68$	C	<i>OK</i>
< 57	D	<i>Poor</i>

Pengujian dilakukan terhadap 18 responden yang diminta mencoba aplikasi Dalsilens secara langsung pada perangkat Android masing-masing selama kurang lebih lima menit, kemudian mengisi kuesioner SUS. Data responden disajikan pada Tabel 3.10.

Tabel 3.10: Data Responden Pengujian SUS

No.	Nama	Usia	Jenis Kelamin
1	Arja Hudaya	20	Laki-laki
2	Ihsan Fadhila	31	Laki-laki
3	Muhamad Adha Ismail	21	Laki-laki
4	Zahra	20	Perempuan
5	Pina	21	Perempuan
6	Mochamad Ariva Alvitto	21	Laki-laki
7	Asep	28	Laki-laki
8	Reza	22	Laki-laki
9	Dina Sartika	20	Perempuan
10	Keiza Aurel	20	Perempuan
11	Syafiq Widyawan Putra	20	Laki-laki
12	Mahez	19	Laki-laki
13	Ghania	19	Perempuan
14	Nur Aulia Rahman	21	Laki-laki
15	Muhammad Arvin Pamungkas	15	Laki-laki
16	Dina Rizkina	20	Perempuan
17	Muhammad Rafi Danendra Putra	21	Laki-laki
18	Saputra	20	Laki-laki

Rekapitulasi jawaban seluruh responden terhadap 10 pernyataan SUS disajikan pada Tabel 3.11.

Tabel 3.11: Rekapitulasi Jawaban Responden Kuesioner SUS

No.	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	Skor SUS
1	2	2	4	1	4	3	5	1	4	2	75,0
2	4	3	4	2	4	3	4	1	3	2	70,0
3	4	2	5	3	3	1	5	1	5	2	82,5
4	4	4	5	2	5	3	5	3	4	3	70,0
5	4	4	4	4	4	3	4	3	4	4	55,0
6	1	1	5	1	5	1	5	1	5	1	90,0
7	4	3	5	3	5	2	5	3	4	4	70,0
8	3	2	4	1	4	3	4	2	3	2	70,0
9	4	3	4	2	4	3	4	4	4	2	65,0
10	3	2	2	3	3	4	2	3	3	2	47,5
11	5	4	5	4	4	3	5	3	5	4	65,0
12	4	3	3	4	4	3	3	4	4	4	50,0
13	2	2	4	1	4	3	5	2	3	2	70,0
14	2	1	5	2	3	3	4	1	3	2	70,0
15	4	3	4	4	3	3	3	2	3	2	57,5
16	2	3	4	2	4	3	5	1	3	4	62,5
17	3	2	4	2	4	2	5	1	4	3	75,0
18	4	2	4	1	5	1	5	1	5	4	85,0
Rata-rata Skor SUS											68,33

Berdasarkan hasil perhitungan menggunakan Persamaan 3.28, diperoleh rata-rata skor SUS sebesar 68,33. Mengacu pada Tabel 3.9, skor tersebut termasuk dalam kategori B (*Good*), yang berarti aplikasi Dalsilens dinilai baik oleh responden dari sisi kebergunaan.

4. PENUTUP

4.1. Kesimpulan

Berdasarkan hasil penelitian dan pembahasan yang telah dilakukan, diperoleh kesimpulan sebagai berikut.

1. Prototipe aplikasi Dalsilens berhasil dikembangkan sebagai alat bantu visual berbasis Android secara *real-time*. Aplikasi mampu mengakuisisi citra dari kamera menggunakan CameraX, memproses setiap *frame* secara langsung, dan menampilkan hasil transformasi warna tanpa jeda yang berarti bagi pengguna. Pengujian fungsionalitas menggunakan metode *Black Box Testing* terhadap 15 skenario menunjukkan seluruh fitur berjalan sesuai ekspektasi dengan hasil **Berhasil** pada semua skenario.
2. Algoritma *LMS Daltonization* berhasil diimplementasikan untuk membantu penderita dikromasi membedakan objek secara visual. Matriks transformasi warna dihitung melalui tiga tahap konversi ruang warna yaitu $RGB \rightarrow LMS \rightarrow$ simulasi defisiensi $\rightarrow RGB$, kemudian diaplikasikan ke *frame* kamera menggunakan *ColorMatrix* Android yang diakselerasi GPU. Fitur ini mendukung tiga jenis dikromasi, yaitu *Protanopia*, *Deutanopia*, dan *Tritanopia*, dengan parameter *severity* yang dapat disesuaikan.
3. Fitur deteksi nama warna berbasis ruang warna HSV berhasil diimplementasikan untuk membantu pengguna membedakan objek secara tekstual. Klasifikasi dilakukan menggunakan pendekatan *rule-based thresholding* yang membagi ruang warna menjadi 10 kategori yaitu tujuh warna kromatik (merah, oranye, kuning, hijau, cyan, biru, ungu) dan tiga warna akromatik (putih, abu-abu, hitam). Hasil pengujian akurasi pada kondisi pencahayaan normal mencapai 90% dan pada kondisi pencahayaan redup mencapai 50%.
4. Pengujian performa *real-time* menunjukkan bahwa aplikasi Dalsilens mampu

mempertahankan rata-rata 57,2 FPS dengan latensi pemrosesan rata-rata 17,5 ms per *frame* saat transformasi *color matrix* dan klasifikasi warna berjalan bersamaan. Penggunaan memori (RAM) tercatat rata-rata sebesar 218,54 MB tanpa menunjukkan tren kenaikan yang konsisten antar iterasi, sehingga tidak ditemukan indikasi *memory leak*. Utilisasi CPU rata-rata hanya sebesar 19,21% dari kapasitas total perangkat, membuktikan bahwa pemindahan beban komputasi dari CPU ke GPU melalui *ColorMatrix* berjalan efektif.

5. Pengujian kebergunaan menggunakan metode *System Usability Scale* (SUS) terhadap 18 responden menghasilkan rata-rata skor sebesar 68,33, yang termasuk dalam kategori B (*Good*) menurut klasifikasi SUS. Hasil ini menunjukkan bahwa aplikasi Dalsilens dinilai baik dan dapat diterima oleh pengguna dari sisi kemudahan dan kenyamanan penggunaan.

4.2. Saran

Berdasarkan penelitian yang telah dilakukan, saran untuk pengembangan selanjutnya adalah sebagai berikut.

1. Fitur klasifikasi nama warna saat ini menggunakan pendekatan *rule-based thresholding* dengan nilai batas yang ditentukan secara empiris. Pengembangan selanjutnya dapat menggunakan pendekatan *machine learning* berbasis data untuk meningkatkan akurasi deteksi, terutama pada kondisi pencahayaan yang berubah-ubah.
2. Aplikasi saat ini hanya mendukung tiga jenis dikromasi (buta warna total pada satu kanal). Pengembangan berikutnya dapat mencakup tipe anomalous trichromacy seperti *Protanomaly* dan *Deutanomaly* yang jumlah penderitanya lebih banyak di populasi umum.
3. Antarmuka pengguna dapat dikembangkan dengan menambahkan mode *accessibility* yang lebih lengkap, seperti umpan balik suara (*text-to-speech*) untuk

nama warna yang terdeteksi, sehingga aplikasi dapat digunakan oleh pengguna dengan keterbatasan visual ganda.

DAFTAR PUSTAKA

- Android Developers. (2024). **CameraX Architecture**.
Diakses dari <https://developer.android.com/media/camera/camerax/architecture>.
- Android Developers. (2026). **Android's Kotlin-first Approach**.
Diakses dari <https://developer.android.com/kotlin/first>.
- Android Developers. (2026). **ColorMatrix | API Reference**.
Diakses dari <https://developer.android.com/reference/android/graphics/ColorMatrix>.
- Android Developers. (2026). **Hardware Acceleration**.
Diakses dari <https://developer.android.com/develop/ui/views/graphics/hardware-accel>.
- Brooke, John. (1995). **SUS: A quick and dirty usability scale**.
Usability Eval. Ind., Volume 189.
- Chung, S. dan Son, J.-W. (2020). **Visual Perception in Autism Spectrum Disorder: A Review of Neuroimaging Studies**.
Soa Chongsonyon Chongsin Uihak, hlm. 105–120.
- Dicoding Indonesia. (2024). **Black Box Testing: Pengertian, Manfaat, dan Cara Kerjanya**.
Diakses dari <https://www.dicoding.com/blog/black-box-testing/>.
- Elrefaei, Lamiaa A. (2018). **Smartphone Based Image Color Correction for Color Blindness**.
International Journal of Interactive Mobile Technologies (iJIM), Volume 12, Nomor 3, hlm. 104–119
Diakses dari <https://online-journaltitles.org/index.php/i-jim/article/view/8160>.
- Francis, William J. (2023). **JetBrains State of Developer Ecosystem 2023: Android and iOS Dominate Mobile Matters**.
Diakses dari <https://www.techrepublic.com/article/jetbrains-state-of-developer-ecosystem-2023-android-ios-mobile/>.

- IBM. (2026). **What is the Software Development Life Cycle (SDLC)?**
Diakses dari <https://www.ibm.com/think/topics/sdlc>.
- JetBrains. (2026). **Kotlin for Android Overview.**
Diakses dari <https://kotlinlang.org/docs/android-overview.html>.
- Lewis, James dan Sauro, Jeff. (2017). **Can I Leave This One Out? The Effect of Dropping an Item From the SUS.**
Journal of Usability Studies, Volume 13, hlm. 38–46.
- Lin, Huei-Yung, Chen, Li-Qi dan Wang, Min-Liang. (2019). **Improving Discrimination in Color Vision Deficiency by Image Re-Coloring.**
Sensors, Volume 19, Nomor 10. ISSN: 1424-8220
Diakses dari <https://www.mdpi.com/1424-8220/19/10/2250>.
- Machado, Gustavo M., Oliveira, Manuel M. dan Fernandes, Leandro A. F. (2009). **A Physiologically-based Model for Simulation of Color Vision Deficiency.**
IEEE Transactions on Visualization and Computer Graphics, Volume 15, Nomor 6, hlm. 1291–1298.
- Man, Dariusz dan Olchawa, Ryszard. (2018). **The Possibilities of Using BCI Technology in Biomedical Engineering.**
Dalam *Advances in Intelligent Systems and Computing*, hlm. 30–37.
- Muhammad, Aisha Badamasi, Maitama, Ja'afar Zubairu, Bature, Amir dan Yunusa, Zainab. (2020). **Analysis of LMS Daltonization Algorithm for Adjusting Image Color for Deuteranopia.**
Zaria Journal of Electrical Engineering Technology, Volume 9, Nomor 1, hlm. 128–135. ISSN: 0261-1570.
- Patterson, E., Mastey, R., Kuchenbecker, J., Rowlan, J., Neitz, J., Neitz, M. dan Carroll, J. (2022). **Effects of color-enhancing glasses on color vision in congenital red-green color deficiencies.**
Optics Express, hlm. 31182–31194.
- RG Guntur Alam dan Puji Rahayu Kurniasih. (2024). **Penggunaan Metode System Usability Scale (SUS) pada Aplikasi SIMAMURAT.**
JSAI: Journal Scientific and Applied Informatics, Volume 7, Nomor 2, hlm. 189–197.
- Samyan, Q.W., Sahar, W., Talha, W., Aslam, M. dan Martinez-Enriquez, A.M. (2015). **Real Time Digital Image Processing Using Point Operations in Multithreaded Systems.**

Dalam *2015 Fourteenth Mexican International Conference on Artificial Intelligence (MICAI)*, hlm. 52–57.

Schmitz, James. (2026). **Color Blindness Simulation Research.**

Diakses dari %5Curl%7Bhttps://ixora.io/projects/colorblindness/color-blindness-simulation-research/%7D.

Sirisathitkul, Yaowarat, Dinmeung, Natthinee, Noonsuk, Wannasan dan Sirisathitkul, Chitnarong. (2025). **Accuracy and precision of smartphone colorimetry: a comparative analysis in RGB, HSV, and CIELAB color spaces for archaeological research.**

STAR: Science & Technology of Archaeological Research, Volume 11, Nomor 1, hlm. e2444168

Diakses dari <https://doi.org/10.1080/20548923.2024.2444168>.

Stack Overflow. (2025). **2025 Stack Overflow Developer Survey.**

Diakses dari <https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language-language>.

StatCounter. (2025). **Mobile Operating System Market Share Worldwide.**

Diakses dari <https://gs.statcounter.com/os-market-share/mobile/worldwide/2025>.

Tasnim, Anika dan Hasan, Mohammad S. (2017). **An improved dynamic daltonization for color-blinds.**

Dalam *2017 IEEE Region 10 Humanitarian Technology Conference (R10-HTC)*, hlm. 798–801.

Tecson, Gerald, Calanda, Fredilyn, Cayabyab, Gerald dan Reyes Jr, Felizardo. (2017). **Covisance: A Real Time Mobile Recolorization Tool...**

Dalam *Proceedings of the 2017 ACM International Conference*, hlm. 95–99.

Viénot, Françoise, Brettel, Hans dan Mollon, John D. (1999). **Digital video colourmaps for checking the legibility of displays by dichromats.**

Color Research and Application, Volume 24, hlm. 243–252

Diakses dari <https://api.semanticscholar.org/CorpusID:62166956>.

LAMPIRAN

Lampiran 1 – Listing Program

A. MainActivity.kt

```
1 class MainActivity : ComponentActivity() {
2
3     private val requestPermissionLauncher = registerForActivityResult(
4         ActivityResultContracts.RequestPermission()
5     ) { isGranted: Boolean ->
6         if (isGranted) hasCameraPermission = true
7         else Toast.makeText(this, "Camera permission is required",
8             Toast.LENGTH_SHORT).show()
9     }
10
11     private var hasCameraPermission by mutableStateOf(false)
12
13     // Navigasi ke layar tujuan, minta izin kamera dulu jika belum diberikan
14     private fun goToScreen(nav: NavHostController, route: String) {
15         if (hasCameraPermission) nav.navigate(route)
16         else requestPermissionLauncher.launch(Manifest.permission.CAMERA)
17     }
18
19     override fun onCreate(savedInstanceState: Bundle?) {
20         super.onCreate(savedInstanceState)
21         enableEdgeToEdge()
22         hasCameraPermission = ContextCompat.checkSelfPermission(
23             this, Manifest.permission.CAMERA
24         ) == PackageManager.PERMISSION_GRANTED
25
26         setContent {
27             DaltonLensTheme {
28                 val navController = rememberNavController()
29                 NavHost(navController, startDestination = "dashboard") {
30                     composable("dashboard") {
31                         DashboardScreen(
32                             onNavigateToCamera = { goToScreen(navController, "camera") },
33                             onNavigateToColorClassification = {
34                                 goToScreen(navController, "color_classification")
35                             }
36                         )
37                     }
38                     composable("camera") { CameraScreen() }
39                     composable("color_classification") {
40                         ColorClassificationTestScreen()
41                     }
42                 }
43             }
44         }
45     }
46 }
```

Listing 4.1: Entry point aplikasi dan konfigurasi navigasi antar layar

B. ColorProcessor.kt

```
1 enum class DeficiencyType(val displayName: String) {
2     Normal("Normal"), Protanopia("Protanopia"),
3     Deuteranopia("Deuteranopia"), Tritanopia("Tritanopia"), Custom("Custom")
4 }
5
6 object ColorProcessor {
7
8     // Konversi RGB ke LMS menggunakan matriks Hunt-Pointer-Estevéz
9     fun rgbToLms(r: Float, g: Float, b: Float): Triple<Float, Float, Float> {
10         val l = 17.8824f * r + 43.5161f * g + 4.11935f * b
11         val m = 3.4565f * r + 27.1554f * g + 3.86714f * b
12         val s = 0.02996f * r + 0.18431f * g + 1.46709f * b
13         return Triple(l, m, s)
14     }
15
16     // Simulasi defisiensi warna dalam ruang LMS
17     fun simulateLms(l: Float, m: Float, s: Float,
18         deficiency: DeficiencyType): Triple<Float, Float, Float> {
19         return when (deficiency) {
20             DeficiencyType.Protanopia -> Triple(2.02344f*m - 2.52581f*s, m, s)
21             DeficiencyType.Deuteranopia -> Triple(l, 0.494207f*l + 1.24827f*s, s)
22             DeficiencyType.Tritanopia -> Triple(l, m, -0.395913f*l +
23                 0.801109f*m)
24             else -> Triple(l, m, s)
25         }
26
27     // Konversi balik LMS ke RGB (invers matriks Hunt-Pointer-Estevéz)
28     fun lmsToRgb(l: Float, m: Float, s: Float): Triple<Float, Float, Float> {
29         val r = 0.080944f * l - 0.130504f * m + 0.116721f * s
30         val g = -0.0102485f * l + 0.0540194f * m - 0.113615f * s
31         val b = -0.000365294f * l - 0.00412163f * m + 0.693513f * s
32         return Triple(r, g, b)
33     }
34
35     // Bangun matriks 4x5 untuk simulasi defisiensi warna
36     fun getDeficiencyMatrix(deficiency: DeficiencyType,
37         severity: Float = 1.0f): FloatArray? {
38         if (deficiency == DeficiencyType.Custom) return null
39         if (deficiency == DeficiencyType.Normal || severity <= 0.0f) {
40             return floatArrayOf(1f,0f,0f,0f,0f, 0f,1f,0f,0f,0f,
41                 0f,0f,1f,0f,0f, 0f,0f,0f,1f,0f)
42         }
43         fun transform(r: Float, g: Float, b: Float): Triple<Float,Float,Float>
44         {
45             val (lN,mN,sN) = rgbToLms(r, g, b)
46             val (lD,mD,sD) = simulateLms(lN, mN, sN, deficiency)
47             return lmsToRgb(lN+(lD-lN)*severity, mN+(mD-mN)*severity,
48                 sN+(sD-sN)*severity)
49         }
50         val (rR,rG,rB) = transform(1f,0f,0f)
51         val (gR,gG,gB) = transform(0f,1f,0f)
52         val (bR,bG,bB) = transform(0f,0f,1f)
53         return floatArrayOf(rR,gR,bR,0f,0f, rG,gG,bG,0f,0f,
54             rB,gB,bB,0f,0f, 0f,0f,0f,1f,0f)
55     }
56
57     // Bangun matriks 4x5 untuk koreksi daltonisasi
58     fun getDaltonizationMatrix(deficiency: DeficiencyType,
59         severity: Float = 1.0f): FloatArray? {
60         if (deficiency == DeficiencyType.Custom ||
61             deficiency == DeficiencyType.Normal) {
```

```

61         return floatArrayOf(1f,0f,0f,0f,0f, 0f,1f,0f,0f,0f,
62             0f,0f,1f,0f,0f, 0f,0f,0f,1f,0f)
63     }
64     fun transform(rA: Float, gA: Float, bA: Float):
Triple<Float,Float,Float> {
65         val (l,m,s) = rgbToLms(rA, gA, bA)
66         val (lS,mS,sS) = simulateLms(l, m, s, deficiency)
67         val (rS,gS,bS) = lmsToRgb(lS, mS, sS)
68         val (eR,eG,eB) = Triple(rA-rS, gA-gS, bA-bS)
69         val (sR,sG,sB) = when (deficiency) {
70             DeficiencyType.Protanopia -> Triple(0f, .9f*eR+.1f*eG,
.9f*eR+.1f*eB)
71             DeficiencyType.Deuteranopia -> Triple(eR+.7f*eG, 0f, .7f*eG+eB)
72             DeficiencyType.Tritanopia -> Triple(eR+.7f*eB, eG+.7f*eB, 0f)
73             else -> Triple(0f, 0f, 0f)
74         }
75         return Triple(rA+sR*severity, gA+sG*severity, bA+sB*severity)
76     }
77     val (rR,rG,rB) = transform(1f,0f,0f)
78     val (gR,gG,gB) = transform(0f,1f,0f)
79     val (bR,bG,bB) = transform(0f,0f,1f)
80     return floatArrayOf(rR,gR,bR,0f,0f, rG,gG,bG,0f,0f,
81         rB,gB,bB,0f,0f, 0f,0f,0f,1f,0f)
82 }
83 }

```

Listing 4.2: Implementasi konversi ruang warna RGB–LMS dan pembangkitan matriks transformasi defisiensi serta daltonisasi

C. ColorNames.kt

```

1 class ColorNames(private val context: Context) {
2
3     // Rata-rata piksel dalam area radius x radius di sekitar titik pusat
4     fun getAverageColor(bitmap: Bitmap, centerX: Int, centerY: Int,
5         radius: Int = 5): Triple<Int, Int, Int> {
6         var r = 0L; var g = 0L; var b = 0L; var count = 0
7         for (dx in -radius..radius) {
8             for (dy in -radius..radius) {
9                 val x = (centerX + dx).coerceIn(0, bitmap.width - 1)
10                 val y = (centerY + dy).coerceIn(0, bitmap.height - 1)
11                 val pixel = bitmap.getPixel(x, y)
12                 r += (pixel shr 16) and 0xFF
13                 g += (pixel shr 8) and 0xFF
14                 b += pixel and 0xFF
15                 count++
16             }
17         }
18         return Triple((r/count).toInt(), (g/count).toInt(), (b/count).toInt())
19     }
20
21     // Klasifikasi nama warna dari nilai RGB menggunakan ruang warna HSV
22     fun getColorNameFromRGB(r: Int, g: Int, b: Int): String {
23         val hsv = FloatArray(3)
24         android.graphics.Color.RGBToHSV(r, g, b, hsv)
25         val h = hsv[0] // Hue : 0.0 - 360.0
26         val s = hsv[1] // Saturation : 0.0 - 1.0
27         val v = hsv[2] // Value : 0.0 - 1.0
28
29         // Deteksi warna akromatik (hitam, abu-abu, putih) berdasarkan saturasi

```

```

30     if (s < 0.10f) {
31         return when {
32             v < 0.15f -> "Black"
33             v < 0.40f -> "Dark Gray"
34             v < 0.78f -> "Gray"
35             else      -> "White"
36         }
37     }
38     if (v < 0.08f) return "Black" // terlalu gelap
39     if (s < 0.18f && v > 0.72f) return "White" // hampir putih
40
41     // Klasifikasi warna kromatik berdasarkan nilai Hue
42     return when {
43         h < 9.85f -> "Red"
44         h < 26.98f -> "Orange"
45         h < 50.42f -> "Yellow"
46         h < 122.50f -> "Green"
47         h < 204.85f -> "Cyan"
48         h < 224.57f -> "Blue"
49         h < 319.37f -> "Purple"
50         else      -> "Red"
51     }
52 }
53
54 fun getCenterColorName(bitmap: Bitmap, radius: Int = 5): String {
55     val (r, g, b) = getAverageColor(bitmap,
56         bitmap.width / 2, bitmap.height / 2, radius)
57     return getColorNameFromRGB(r, g, b)
58 }
59 }
60
61 data class ColorItem(val hex: String, val name: String)

```

Listing 4.3: Implementasi klasifikasi nama warna berbasis ruang warna HSV dengan pendekatan *rule-based thresholding*

D. CameraScreen.kt (bagian akuisisi citra)

```

1 // Rotasi bitmap sesuai orientasi sensor kamera
2 fun rotateIfNeeded(bitmap: Bitmap, rotation: Int): Bitmap {
3     if (rotation == 0) return bitmap
4     val matrix = Matrix().apply { postRotate(rotation.toFloat()) }
5     return Bitmap.createBitmap(bitmap, 0, 0,
6         bitmap.width, bitmap.height, matrix, true)
7 }
8
9 // Analisis satu frame: rotasi, deteksi warna, lalu update state
10 fun analyzeFrame(imageProxy: ImageProxy) {
11     try {
12         val bitmap = imageProxy.toBitmap()
13         val rotation = imageProxy.imageInfo.rotationDegrees
14         val finalBitmap = rotateIfNeeded(bitmap, rotation)
15
16         centerColorName = colorNames
17             .getCenterColorName(finalBitmap, sampleAreaSize)
18         processedBitmap = finalBitmap
19     } catch (e: Exception) {
20         Log.e("CameraScreen", "Error processing frame", e)
21     } finally {
22         imageProxy.close()

```

```

23     }
24 }
25
26 // Inisialisasi CameraX dan pipeline ImageAnalysis
27 LaunchedEffect(Unit) {
28     val cameraProvider = context.getCameraProvider()
29     val resolutionSelector = ResolutionSelector.Builder()
30         .setAspectRatioStrategy(RATIO_16_9_FALLBACK_AUTO_STRATEGY)
31         .build()
32
33     val preview = Preview.Builder()
34         .setResolutionSelector(resolutionSelector).build()
35         .also { it.setSurfaceProvider(previewView.surfaceProvider) }
36
37     val imageAnalysis = ImageAnalysis.Builder()
38         .setResolutionSelector(resolutionSelector)
39         .setBackpressureStrategy(STRATEGY_KEEP_ONLY_LATEST)
40         .build()
41         .also { it.setAnalyzer(analysisExecutor, ::analyzeFrame) }
42
43     cameraProvider.unbindAll()
44     cameraProvider.bindToLifecycle(
45         lifecycleOwner,
46         CameraSelector.DEFAULT_BACK_CAMERA,
47         preview,
48         imageAnalysis
49     )
50 }
51
52 // Terapkan ColorFilter dari matriks transformasi ke bitmap hasil kamera
53 if (isFilterEnabled) {
54     processedBitmap?.let { bitmap ->
55         Image(
56             bitmap = bitmap.asImageBitmap(),
57             contentDescription = null,
58             modifier = Modifier.fillMaxSize(),
59             contentScale = ContentScale.Crop,
60             colorFilter = ColorFilter.colorMatrix(ColorMatrix(matrixArray))
61         )
62     }
63 }

```

Listing 4.4: Inisialisasi CameraX dan loop analisis frame secara *real-time*

E. DashboardScreen.kt

```

1  @Composable
2  fun DashboardScreen(
3      onNavigateToCamera: () -> Unit,
4      onNavigateToColorClassification: () -> Unit
5  ) {
6      Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
7          Column(
8              modifier = Modifier.fillMaxSize().padding(innerPadding),
9              verticalArrangement = Arrangement.Center,
10             horizontalAlignment = Alignment.CenterHorizontally
11         ) {
12             Text(text = "Dashboard")
13             Button(
14                 onClick = onNavigateToCamera,

```

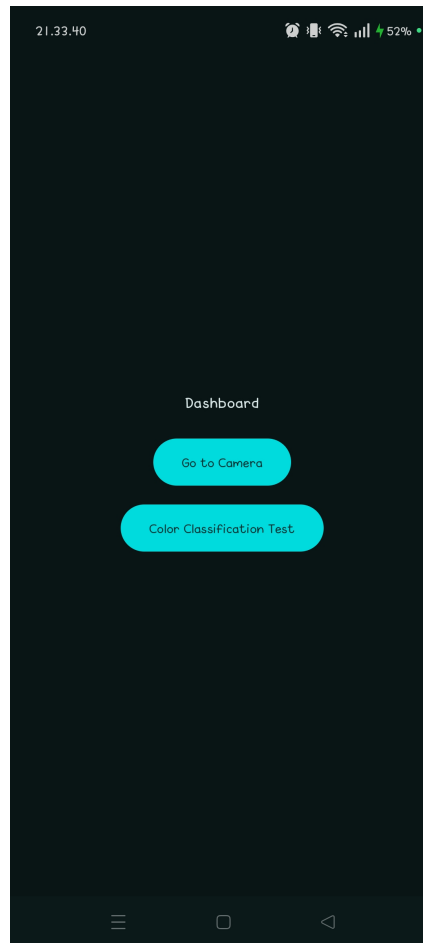
```

15         modifier = Modifier.padding(top = 16.dp)
16     ) { Text(text = "Go to Camera") }
17     Button(
18         onClick = onNavigateToColorClassification,
19         modifier = Modifier.padding(top = 8.dp)
20     ) { Text(text = "Color Classification Test") }
21 }
22 }
23 }

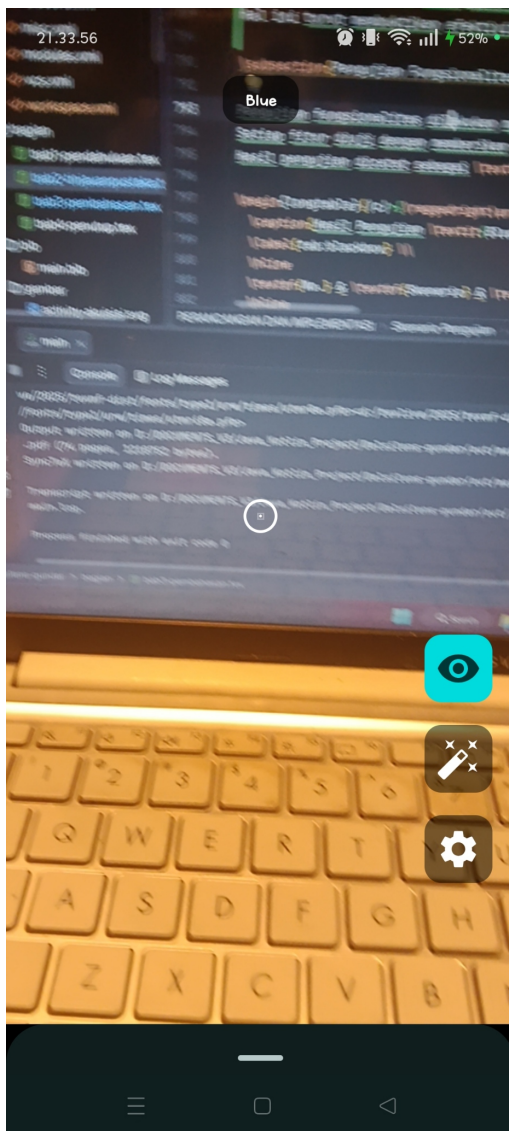
```

Listing 4.5: Implementasi halaman utama (*dashboard*) dengan navigasi ke fitur kamera dan uji klasifikasi warna

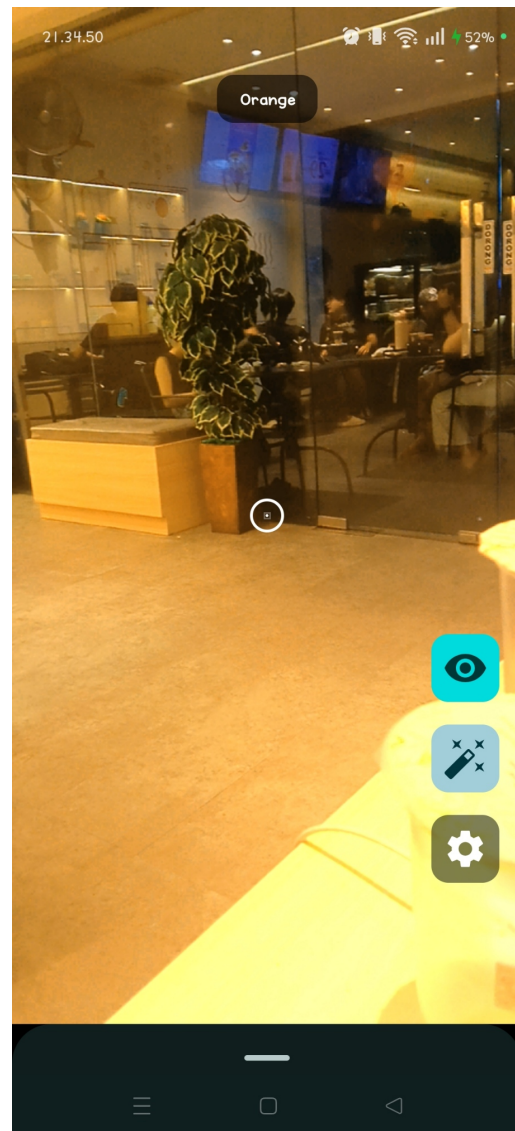
Lampiran 2 – Output Program



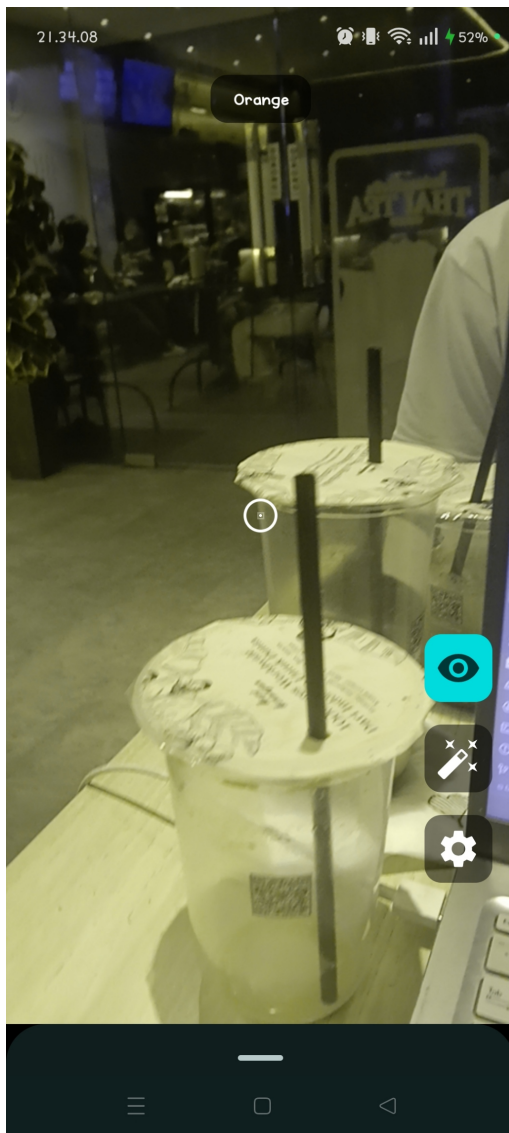
Gambar L-3.1. Halaman Dashboard



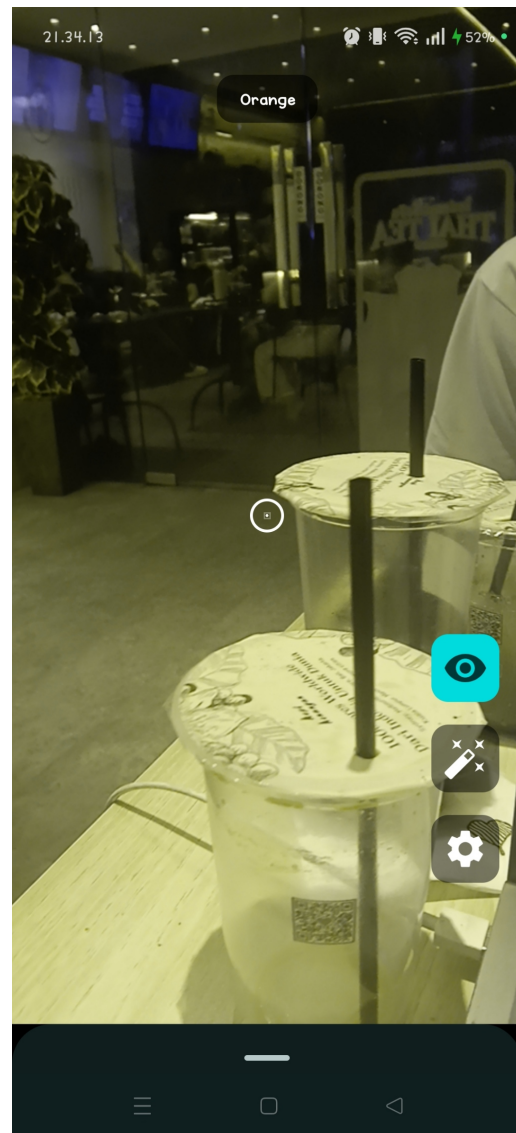
Gambar L-3.2. Tampilan Mode Normal



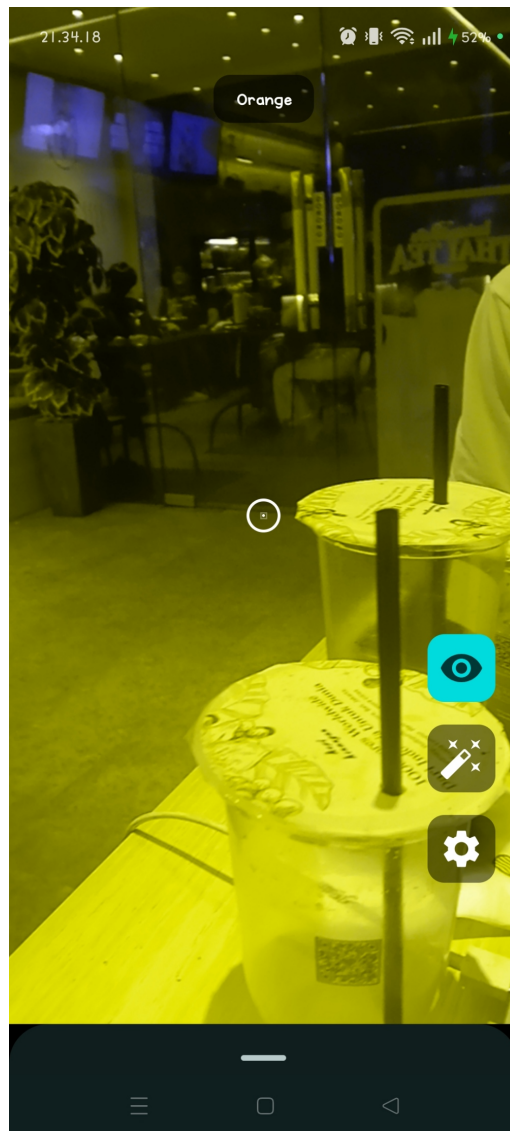
Gambar L-3.3. Tampilan Mode Daltonisasi



*Gambar L-3.4. Tampilan Simulasi
Protanopia*



*Gambar L-3.5. Tampilan Simulasi
Deuteranopia*



Gambar L-3.6. Tampilan Simulasi Tritanopia